pyAmpliCol at one loop

The recursive one-loop engine against MadLoop, Recola, OpenLoops and NJet

milestone 2026-10-26 · engine `4fc72e39` on `loop` · AMD EPYC 9754, shared host · finite part · binary64 · one pinned thread · batches of $B = 128$

pyAmpliCol timed by G1 on 2026-10-25: engine `1d27426b`, native `loop-native-v2-slice-7db32901` (not re-measured on `4fc72e39`)

×1/16 faster — ×1/4 — ×1 run time ÷ fastest provider — ×4 — ×16 slower

Gate pass ×0.25 · fail ×1.4 · open ×1.0 · <5 ms: not gated ×1.4 · time ÷ limit

Stages inputs · numerator · scatter · NinjaRS · masters · glue

missing (why) · no colour: generation, LC † within estimate ¶ reference's own (DD) ‡ user decision

ML MadLoop · Rc Recola · OL OpenLoops

Table 1: **State of the one-loop engine.** How to read it: after Figure 1.

	Process · validation	Generation	Full colour · per point, batched	Leading colour (LC)
pyAmpliCol values: the most recent measured configuration — `loop` · $2 \to 2$ to $2 \to 5$: 1d27426b, native per-point inputs, G1 2026-10-25				
	legs · corrections · loop families; largest deviation of the finite part per reference	generate + plan export, peak memory, plan size; provider generation on this host	median per point at the largest batch; provider times; gate G1 (time ÷ limit); stage shares	batched, same configuration; LC/FC; no provider ratio
$2 \to 2$	$gg \to gg$ 4 legs · QCD · 24 families (LC 12) Rc·OL·ML ≤ 1.8e−12 16 pts · median 1.7e−14 · †	**9.04 s** ×64 0.38 GiB peak · plan 0.305 MB **Rc 142 ms** · ML 9.70 s	**1.56 ms** ×0.77 **ML 2.02 ms** · OL 3.20 ms · Rc 10.0 ms $B = 128$ no gate (2→2) numerator 56 %	**0.716 ms** LC/FC 0.46 $B = 128$ · FC 1.56 ms
$2 \to 3$	$u\bar{u} \to d\bar{d}g$ 5 legs · QCD · 42 families (LC 27) ML·OL ≤ 6.7e−13 · Rc 3.3e−10 16 pts · median 8.7e−14 · ¶	**8.96 s** ×22 0.39 GiB peak · plan 0.282 MB **Rc 406 ms** · ML 9.87 s	**1.91 ms** ×1.3 **OL 1.52 ms** · ML 3.17 ms · Rc 3.18 ms $B = 128$ <5 ms: not gated ×0.97 numerator 40 %	**1.28 ms** LC/FC 0.67 $B = 128$ · FC 1.91 ms
	$u\bar{u} \to u\bar{u}g$ 5 legs · QCD · 68 families (LC 42) OL·ML ≤ 8.4e−12 · Rc 7.2e−11 16 pts · median 1.7e−13 · ¶	**9.13 s** ×20 0.39 GiB peak · plan 0.388 MB **Rc 462 ms** · ML 10.9 s	**2.65 ms** ×0.91 **OL 2.92 ms** · Rc 5.55 ms · ML 8.59 ms $B = 128$ <5 ms: not gated ×0.70 numerator 46 %	**1.64 ms** LC/FC 0.62 $B = 128$ · FC 2.65 ms
	$u\bar{u} \to e^-e^+g$ 5 legs · QCD · 11 families (LC 7) Rc·ML ≤ 1.3e−13 16 pts · median 9.6e−15 · †	**10.7 s** ×15 0.43 GiB peak · plan 0.310 MB **Rc 701 ms** · ML 14.0 s	**0.716 ms** ×0.76 **ML 0.948 ms** · Rc 1.08 ms · OL no library $B = 128$ <5 ms: not gated ×0.76 numerator 37 %	**0.521 ms** LC/FC 0.73 $B = 128$ · FC 0.716 ms
	$u\bar{u} \to ggg$ 5 legs · QCD · 90 families (LC 62) ML·Rc ≤ 1.5e−12 · OL 8.4e−11 16 pts · median 7.2e−14 · ¶	**10.4 s** ×16 0.41 GiB peak · plan 0.656 MB **Rc 634 ms** · ML 17.6 s	**5.17 ms** ×0.42 **OL 12.4 ms** · Rc 25.3 ms · ML 28.2 ms $B = 128$ pass ×0.32 numerator 57 %	**2.70 ms** LC/FC 0.52 $B = 128$ · FC 5.17 ms
	$gg \to ggg$ 5 legs · QCD · 154 families (LC 77) ML·Rc·OL ≤ 7.6e−7 16 pts · median 6.1e−13 · †	**17.0 s** ×2.6 0.50 GiB peak · plan 1.85 MB **Rc 6.63 s** · ML 47.0 s	**20.2 ms** ×0.21 **OL 97.0 ms** · ML 247 ms · Rc 351 ms $B = 128$ pass ×0.16 numerator 59 %	**8.27 ms** LC/FC 0.41 $B = 128$ · FC 20.2 ms
$2 \to 4$	$u\bar{u} \to e^-e^+gg$ 6 legs · QCD · 69 families (LC 33) Rc·ML ≤ 1.2e−10 16 pts · median 5.9e−14 · †	**12.0 s** ×14 0.44 GiB peak · plan 0.624 MB **Rc 833 ms** · ML 24.8 s	**4.98 ms** ×0.49 **Rc 10.2 ms** · ML 27.3 ms · OL no library $B = 128$ <5 ms: not gated ×0.37 numerator 57 %	**1.71 ms** LC/FC 0.34 $B = 128$ · FC 4.98 ms
	$gg \to t\bar{t}gg$ 6 legs · QCD · 1 086 families (LC 375) OL·Rc·ML ≤ 1.3e−8 16 pts · median 1.6e−11 · †	**31.5 s** ×0.51 0.66 GiB peak · plan 3.71 MB **Rc 61.4 s** · ML 218 s	**99.4 ms** ×0.21 **OL 472 ms** · Rc 1.97 s · ML 5.45 s $B = 128$ pass ×0.16 numerator 48 %	**33.3 ms** LC/FC 0.33 $B = 128$ · FC 99.4 ms
	$u\bar{u} \to gggg$ 6 legs · QCD · 628 families (LC 375) Rc 9.7e−12 · OL 2.0e−10 16 pts · median 1.5e−12 · †	**32.0 s** ×0.97 0.70 GiB peak · plan 4.54 MB **Rc 32.8 s**	**79.0 ms** ×0.21 **OL 374 ms** · Rc 908 ms · ML not exported $B = 128$ pass ×0.16 numerator 62 %	**29.8 ms** LC/FC 0.38 $B = 128$ · FC 79.0 ms
	$gg \to gggg$ 6 legs · QCD · 1 100 families (LC 550) ML·Rc·OL ≤ 2.4e−7 8 pts · median 2.7e−11 · †	**122 s** ×0.058 1.85 GiB peak · plan 16.1 MB **Rc 35.4 min** · ML 144.6 min	**397 ms** ×0.14 **OL 2.84 s** · Rc 17.6 s · ML 38.7 s $B = 128$ pass ×0.11 numerator 56 %	**158 ms** LC/FC 0.40 $B = 128$ · FC 397 ms

	Process · validation	Generation	Full colour · per point, batched	Leading colour (LC)
pyAmpliCol values: the most recent measured configuration — loop · $2 \to 2$ to $2 \to 5$: 1d27426b, native per-point inputs, G1 2026-10-25				
	legs · corrections · loop families; largest deviation of the finite part per reference	generate + plan export, peak memory, plan size; provider generation on this host	median per point at the largest batch; provider times; gate G1 (time ÷ limit); stage shares	batched, same configuration; LC/FC; no provider ratio
$2 \to 5$	$u\bar{u} \to e^-e^+d\bar{d}g$ 7 legs · QCD · 242 families (LC 114) ML·OL·Rc ≤ 5.7e−11 16 pts · median 8.0e−13 · †	**17.1 s** ×2.5 0.49 GiB peak · plan 1.13 MB **Rc 6.73 s** · ML 58.6 s	**16.0 ms** ×0.45 **Rc 35.3 ms** · OL 51.0 ms · ML 132 ms $B = 128$ pass ×0.35 numerator 55 %	**6.13 ms** LC/FC 0.38 $B = 128$ · FC 16.0 ms
	$u\bar{u} \to e^-e^+ggg$ 7 legs · QCD · 463 families (LC 164) ML·Rc·OL ≤ 1.1e−10 16 pts · median 1.5e−12 · †	**16.8 s** ×1.3 0.52 GiB peak · plan 2.45 MB **Rc 13.4 s** · ML 146 s	**55.6 ms** ×0.22 **Rc 251 ms** · OL 346 ms · ML 1.42 s $B = 128$ pass ×0.17 numerator 69 %	**10.4 ms** LC/FC 0.19 $B = 128$ · FC 55.6 ms
	$gg \to t\bar{t}e^-e^+g$ 7 legs · QCD · 579 families (LC 164) Rc·OL·ML ≤ 8.7e−9 16 pts · median 2.7e−11 · †	**22.7 s** ×0.82 0.60 GiB peak · plan 2.13 MB **Rc 27.7 s** · ML 165 s	**60.6 ms** ×0.15 **OL 404 ms** · Rc 620 ms · ML 3.04 s $B = 128$ pass ×0.12 numerator 63 %	**11.1 ms** LC/FC 0.18 $B = 128$ · FC 60.6 ms
	$gg \to t\bar{t}e^-e^+g$ 7 legs · EW · families not recorded **in development (WP3.2): massive EW loops (EWI-4b), lepton lines (EWI-5), EW native inputs**	***in development (WP3.2)*** massive EW loops (EWI-4b), lepton lines (EWI-5), EW native inputs: no configuration generates it yet **Rc 21.9 s** · ML 10.9 min	***in development (WP3.2)*** production, s/pt (20 Oct): OL 0.96–1.12 · Rc 0.95–1.02 · ML 6.90–7.08 gate limit 1.235 s/pt (Recola)	***in development (WP3.2)***
	$u\bar{u} \to gggg$ 7 legs · QCD · 5 258 families (LC 2 712) ML 2.1e−12 · Rc 2.9e−10 3 pts · median 7.1e−11 · ¶	**368 s** ×0.036 4.80 GiB peak · plan 69.2 MB **Rc 170.3 min** · ML 12.2 h	**1.66 s** ≥×0.0091 **Rc ≤182 s** · ML 569 s · OL no library $B = 128$ pass ×0.0029 numerator 68 %	**522 ms** LC/FC 0.31 $B = 128$ · FC 1.66 s
	$gg \to gggg$ 7 legs · QCD · 4 707 families (LC 4 707) NJet DD 3.7e−8 10 pts · median 2.0e−9 · † · top-free (particle_veto = [t]), Nf = 5 ‡ pts 4, 8 above 2.0e−8: accepted (user 2026-10-25)	**43.7 min** 23.0 GiB peak · plan 160 MB ML diagrams 17.4 h, stopped in export · Rc unfinished	**6.97 s** $B = 128$ no gate (no matched provider) numerator 51 %	**3.47 s** LC/FC 0.50 $B = 128$ · FC 6.97 s

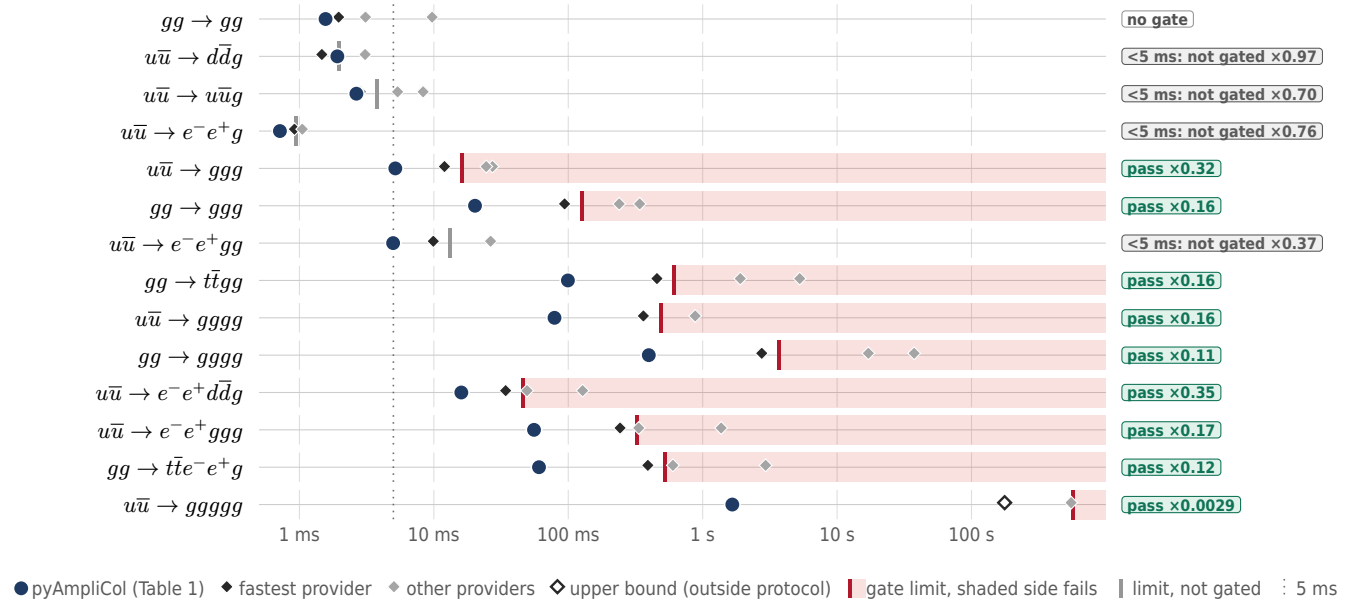


Figure 1: **Batched full colour against the gate** (finite part, $B = 128$; the rows of Table 1 with a matched provider).

Reading the summary

Table 1 has one row per test process, by multiplicity. A cell gives pyAmpliCol's value in bold and, at its right, the multiplier *pyAmpliCol ÷ the fastest provider* on this host (in bold among the provider values). A provider time marked ≤ ($u\bar{u} \to ggggg$) comes from a run outside the protocol that does the production work and more (Recola's reference run evaluates the IR poles too), so it bounds that provider's production time from above, and the multiplier against it is a lower bound (≥); it is never a gate limit. Only run times are coloured: green faster, white at parity, red slower ($\log_2$ scale, clamped at ×1/16 and ×16). Generation cells carry their multiplier uncoloured (generation time is not the main metric, user 2026-10-24); leading colour has no provider ratio. Grey cells say why a value is missing. Run times are per phase-space point in batches, finite part, stability checks off; providers in production mode (Section 1). Every value is the **most recent measured configuration** of its cell (by git ancestry of the revisions; a newer check that only repeats part of an older comparison does not replace it): $2 \to 2$ to $2 \to 5$: the engine on `loop` with native per-point inputs, gate G1 at `1d27426b` (2026-10-25). The engine before G-int (Python per-point inputs, G1 at `1c971289`) is history in Section 2. The column header of Table 1 names these configurations once.

Gate chips (Table 1 and Figure 1): the time over the binding limit (MadLoop's time, or 1.30 × the faster of Recola and OpenLoops), **pass** or **fail** when the whole uncertainty band is below or above 1, else **open**. The user's gate rule of 2026-10-24: a process whose batched time per point is below 5 ms has no blocking gate, its verdict is "below 5 ms: not gated (user 2026-10-24)" and its chip ("<5 ms: not gated") keeps the measured ratio ($u\bar{u} \to d\bar{d}g$ 1.91 ms, 0.97 of its limit, $u\bar{u} \to u\bar{u}g$ 2.65 ms, 0.70 of its limit, $u\bar{u} \to e^-e^+g$ 0.716 ms, 0.76 of its limit and $u\bar{u} \to e^-e^+gg$ 4.98 ms, 0.37 of its limit). $gg \to gg$ ($2 \to 2$) has no gate.

Stage shares (the bar of the full-colour cell; the largest is named; Table 4): inputs = trees, Born and counterterm inputs (on `loop` the executor's own timer of its native trees, since lc-perf); glue = point parameters, input conversion, assembly and the rest of the call. Times use the protocol's schedule B.

References and deviations. ML MadLoop 3.7.0, Rc Recola 2.2.5 (COLLIER), OL OpenLoops 2.1.5 (prebuilt libraries: no generation time), NJet DD NJet 3.1.2 in double-double. Deviation $|a-b| \,/\, \max(|a|, |b|, |B|\ \alpha_s/2\pi)$ of the binary64 finite part at the timed points (median: over the points, the largest over the references); † the largest lies within the binary64 stability estimate of the same configuration at that point; ¶ the largest is the reference's own: pyAmpliCol's double-double value at that point lies as far from it, and its binary64 value lies within E of the double-double one. ‡ $gg \to ggggg$: the reference check's points 4 and 8 (3.7×10^{-8} and 2.2×10^{-8} from NJet DD) exceed its tolerance 2×10^{-8} in binary64: **accepted (user 2026-10-25): O(1e-8) binary64, within the estimate, DD = NJet** (estimates E 1.3×10^{-6} and 2.5×10^{-7}; pyAmpliCol's double-double values equal the reference's); the check keeps its tolerance and records `passed: false` (`data/milestone.json` `validation_decisions`).

Configurations (main-configuration rule of `scripts/build_summary.py`; `data/summary.json` names the configuration and source of every value). $2 \to 2$ to $2 \to 5$: the engine on `loop`, gate G1 at `1d27426b`, build `loop-native-v2-slice-7db32901`, native per-point inputs (Section 2). Generation excludes model loading; unbatched development timings (e.g. a seven-gluon point with its rotation check) are not in this table (Section 4).

Figure 1 places each row's batched full-colour value of Table 1 (dots; the most recent measured configuration: $2 \to 2$ to $2 \to 5$: `1d27426b`, native per-point inputs, G1 2026-10-25) against the providers (diamonds, the darkest the fastest) and the gate limit (red bar, the shaded side fails; grey bar: the limit of a process below 5 ms, not gated; dotted line: 5 ms), for the rows with a matched provider, at $B = 128$. The ratios and bands at every batch size: Table 3.

1 Status at this milestone

On loop (engine 4fc72e39; merges since bae3b9ea, engine 1d27426b)

1b13005a	native-dag	**A4: native DAG route.**
6c87e276	ew-gauge	**EWI-4a: Feynman-gauge trees.**
a8409830	precision-ap	**On-shell master invariants.**
392d3f4a	ew-ct	**EWI-0, EWI-3: EW counterterms.**
46f72f20	measure-g1-7leg	**G1 at 2→5.**
48aeb123	reference-relocate	**Relocated provider trees.**
4e9262b5	model-determinism	**Deterministic models.**
79f2a5bd	native-lowering	**D phase 1 part 1: Rust plan writer.**
4fc72e39	onshell-consistency	**Binary64-miss re-check.**

Before (engine 1d27426b): v2 waves 0-1 (gates S1, G0), native executor, colour, stability API, schedule M, observables, NJet reference, public v2 engine with plan v4 artifacts, counterterms as plain tree insertions (addendum J), runtime pole selection, Rust A0-A3, the G1 harness, G-int (native per-point inputs and generation stages), the summary-first report; at milestone m1025a ct-perf, ew-poles (EW guards, QED Catani operator), lc-perf, artifact-portability and ew-lines. **Native build** `loop-native-v2-slice-79f2a5bd` (digest `2ba11490`…); G1 ran on `loop-native-v2-slice-7db32901` (engine `1d27426b`).

Gate G1 (Section 2; finite part, $2 \to 3$ and beyond)

- **Most recent configuration** (Table 1 and Figure 1): native per-point inputs on `loop` (G1 at `1d27426b`) at $B = 128$: 9 of 9 gated processes pass; $u\bar{u} \to d\bar{d}g$, $u\bar{u} \to u\bar{u}g$, $u\bar{u} \to e^-e^+g$ and $u\bar{u} \to e^-e^+gg$ run below 5 ms per point: **not gated** (user 2026-10-24; 0.97, 0.70, 0.76, 0.37 of the limit). At $B = 1$ no gated process passes with 16 lanes (5 with one lane: the development runs of `39824f9b`, history; the public path runs 16 lanes).
- **$2 \to 5$** (new on `loop` with this milestone, package `measure-g1-7leg`; full and leading colour at $B = 16$ and 128): 4 of 4 gated rows pass at $B = 128$ ($u\bar{u} \to e^-e^+d\bar{d}g$ 16.0 ms, 0.35 of its limit; $u\bar{u} \to e^-e^+ggg$ 55.6 ms, 0.17 of its limit; $gg \to t\bar{t}e^-e^+g$ 60.6 ms, 0.12 of its limit; $u\bar{u} \to ggggg$ 1.66 s, 0.0029 of its limit, ≥×0.0091 against Recola's upper bound); $gg \to ggggg$ 6.97 s at full and 3.47 s at leading colour per point, no matched provider and no gate, Ward check pending — needs native Ward inputs.
- **Before G-int** (Python per-point inputs, G1 at `1c971289`; history, Section 2): failed every gated process it generated, at ×11–20 of the binding limit at $B = 128$ (the Python inputs took 87 %–99 % of the time) and at ×12–27 for $gg \to gggg$, $gg \to t\bar{t}gg$ and $u\bar{u} \to gggg$, timed at $B = 1$ only, and refused $u\bar{u} \to e^-e^+g$ and $u\bar{u} \to e^-e^+gg$.

In development (branches, not on `loop`; HANDOFF §6)

- **Rust phase 1** (HANDOFF §0.4): A0–A4 are on `loop`, A4 switched on (the native DAG builder is the production route for the loop blocks and the counterterm program; counterterm-route parity with the site oracle met), and D's phase 1 part 1 (`native-lowering`: the Rust plan writer, the bounded binary plan encoding, D1 leaf weights). Next the switches of seeds (B, `native-seeds`) and cells (C, `native-cells`), both under audit, and D3, the plan lowering in Rust (about 55 s at seven gluons, target under 15 s).
- **generation-scale-2** (under audit): every stored-process call without regeneration (`"uv"`/`"ir"`, correlated, resolved), SDK v4 runtime, native Catani operator, native Ward inputs (the seven-gluon Ward check), the binary plan member in artifacts, and the genericity audit's M1 and M2 (right).
- **Electroweak** (WP3.2): W, Z, Goldstone and ghost loop lines in Feynman gauge with the Goldstone–vector mixing counterterms (EWI-4b, `ew-gauge`, in progress); the counterterm lowering within the plan's 30 s (EWI-3R; 259 s cold in measurement M1); then lepton-line signs (EWI-5, `ew-leptons`, free to start since `native-lowering`), EW native inputs (EWI-N), the EW ladder and the target (EWI-7, EWI-9), the complex-mass scheme (EWI-10).
- **Genericity:** `genericity-fixes` (in a fix round): M3, and the audit's renamed-model harness (renamed orders, particles, PDG codes and parameters as a durable test).

Missing or incomplete

- Electroweak loops (WP3.2, in development): $gg \to t\bar{t}e^-e^+g$ (EW) is not generated; it needs the massive EW loop lines (EWI-4b, in progress), lepton-line signs (EWI-5) and EW native inputs. On `loop`: typed refusals of EW requests with a non-zero width or an external photon, the QED Catani operator, EW fermion lines (more than one colour-singlet fermion line refused until EWI-5), Feynman-gauge trees (EWI-4a) and the UFO's EW counterterms with MadLoop's branch-side convention (EWI-0, EWI-3).
- **Genericity audit** (DECISIONS 2026-10-26 ~00:20; no process-keyed branch found), three majors, their fixes not yet on `loop`: **M1** any `NotImplementedError` of the native tree export silently makes an artifact plan-only (`regenerated-at-load`), to become a typed, recorded refusal (generation-scale-2); **M2** derived (internal) masses are not inputs of the native parameter program (e.g. M_W of `loop_sm`), so such processes take M1's route with p^2 invariants (generation-scale-2); **M3** generation upper-cases coupling-order names while the loop request scope matches the model's spelling, so a model with lower-case order names is refused (`genericity-fixes`).
- Gate G1 at $B = 1$ (16 lanes for one point); provider protocol-mode columns.
- Seven gluons: Ward check pending — needs native Ward inputs (coordinator ruling of 25 October: generation-scale-2's next run; with Python per-point Ward inputs one evaluation is estimated at 46–69 h). Generation of $gg \to ggggg$ at full colour: 43.7 min with the plan export, peak 23.0 GiB (Section 4), inside addendum H, to be made lean; its LC plan keeps all 4 707 families and generates more slowly (Section 2.2; investigation: `native-cells`).
- Prepared artifacts (addendum G): relocatable since `7db32901` (no host strings, SymJIT applications recompiled at load, bitwise binary64 after relocation), but correlated, resolved and flavour calls, the `"uv"`/`"ir"` selections and the plan-only artifacts of M1 still regenerate on first use (generation-scale-2), and the parameter program's SymJIT bytes differ per compilation (values unaffected; upstream fix filed).

Performance gates and timing protocol: definitions

- **Gate.** Faster than MadLoop and at most 1.30× each matched Recola and OpenLoops time, on the **finite part**, from $2 \to 3$ upward; $2 \to 2$ is reported without a gate. A process whose batched time per point is below 5 ms has no blocking gate: its ratio is shown, its verdict is "below 5 ms: not gated (user 2026-10-24)" (user decision; `data/milestone.json` `gate_decisions.threshold_rule`); optimisation effort goes to higher multiplicities. Aspiration: fastest of all tools from $2 \to 3$.
- **Protocol.** Providers in production mode with poles off where they allow it (OpenLoops `truepoles 0`, Recola without IR poles, MadLoop production build); pyAmpliCol finite-only (`poles = "none"`) with stability checks off, one pinned thread, binary64, 16 lanes, the same frozen points (pyAmpliCol receives their exact rationals, the providers their binary64 decimal text); since milestone m1025a (G1 at `1d27426b`) pyAmpliCol's setup heap is frozen out of Python's cyclic collector before the warm-up (a loaded runtime holds no generation objects; in the committed harness since `de8d3eb8`; Section 2). Statistic: median per point; the uncertainty band combines pyAmpliCol's block spread and the provider's repeat spread; a gate whose band contains the limit is **open**.
- **Timing boundary.** pyAmpliCol: the public v2 runtime call (`NativeExecutor.evaluate`) for a batch of B distinct points, everything included (per-point inputs, loop stage, assembly); the public `Runtime.evaluate` of a loaded artifact adds its input parsing (Section 2). Providers have no batch API: one time per point serves every B. Generation: pyAmpliCol `LoopSlice.generate` plus the plan export, in G1 on one idle pinned core; a development record's generation in Table 1 ran on the shared host under load (threads per increment: Table 12), without a multiplier in a G1 row; MadLoop graph generation, export and compilation; Recola process generation; OpenLoops libraries are prebuilt (no generation time).
- **Labels.** Table 1 and Figure 1 show the most recent measured configuration of each cell (main-configuration rule of `scripts/build_summary.py`; `data/summary.json` names the configuration and source of every value); the column header of Table 1 states it once, with branch and commit, and "Reading the summary" says when it is a development measurement not yet on `loop`. The detailed G1 tables (Section 2) show the configuration on `loop` and, labelled, its history: the G-int runs (`4e9b5a9e`), the Python per-point inputs before G-int and the development runs of `39824f9b`. Missing values are marked, never estimated or extrapolated; test durations are never used as event timings. The legacy per-diagram engine's timings of 18 October are history (data kept in `data/measurements.json`, not shown).

2 Gate G1: end-to-end timings of the v2 engine

G1 times the complete Born–virtual evaluation, **finite part only**, through the call of the public v2 runtime (`NativeExecutor.evaluate`, which `V2VirtualRuntime` calls) with stability checks off, on one pinned core and one thread, binary64, 16 SIMD lanes, in batches of B distinct points of a frozen stream (REQUIREMENTS_ADDENDUM D, plan §5.8); pyAmpliCol receives the stream's exact rational points, the providers their binary64 decimal text. The configurations:

- **native inputs, on `loop`** — the engine at `1d27426b` (milestone m1025a: G-int with ct-perf, ew-poles, lc-perf, artifact-portability and ew-lines; build `loop-native-v2-slice-7db32901`): the executor forms the trees, the Born and the counterterm inputs per point, with binary64 SymJIT plane applications for every native tree kernel and the parameter program (`7db32901`) and tree closures in lanes of points (`88657260`; its report times them), then runs the loop stage. A stored artifact written by the public generator holds the plan v4 with `point_inputs = "native-v2"` (the SymJIT applications stored as IR, recompiled at load); for $u\bar{u} \to d\bar{d}g$ its plan digest equals the harness's (and G-int's), and `Runtime.load` in a fresh interpreter with generation forbidden gives the native executor (16 lanes), regenerates nothing over 128 evaluations, and returns bitwise the timed run's binary64 values at all 128 stream points. This milestone did not re-time it on the engine now on `loop`, `4fc72e39` (native-dag A4, ew-gauge EWI-4a, precision-ap, ew-ct EWI-0/EWI-3, reference-relocate, model-determinism, native-lowering D phase 1 part 1 and onshell-consistency; build `loop-native-v2-slice-79f2a5bd`). `measure-g1-7leg`'s integration checked the $2 \to 5$ rows on its head `db13c260` (build `loop-native-v2-slice-a8409830`; $B = 16$, one pinned core; $u\bar{u} \to e^-e^+d\bar{d}g$ at full and leading colour, $gg \to t\bar{t}e^-e^+g$ and $u\bar{u} \to ggggg$ at full colour, $u\bar{u} \to e^-e^+ggg$ at leading colour; not timing runs): bitwise the timed binary64 values at 16 of 16 points in every job. The four merges after it (reference-relocate, model-determinism, native-lowering, onshell-consistency) were not checked on the G1 rows; their records report the plan JSON byte-identical and the numerators bitwise equal (native-lowering), and plans and full-colour values of $gg \to gg$ and $u\bar{u} \to d\bar{d}g$ bitwise equal while binary64 model parameters moved by at most 8 ulp, EW values in their last bits (model-determinism). Its binary64 values equal those of G-int (`4e9b5a9e`) and of the development campaign of `39824f9b` (`work/v2-generation-scale` `889af02c` plus timers), whose runs are its earlier runs, to rounding level (median relative change 3×10^{-15} to 3×10^{-11} per process; binary64-unstable points move more, about 10^{-3} at $gg \to ggg$ point 66 and $gg \to gggg$ point 50, outside the provider comparison); the development campaign's one-lane, schedule-M and stability runs were not repeated and appear below as history, labelled `39824f9b`;
- **Python inputs** history — the engine on `loop` before G-int, G1 at `1c971289` (LC at `d591a2f7`): trees, Born amplitudes and counterterm scalars formed per point in Python, then the loop stage natively. It refused the V+jets rows: their closed-quark Agg triangles needed Furry's certified zeros (WP3.5), and their Z couplings on loop quark lines the electroweak fermion signs (WP3.2).

Timing protocol at `1d27426b` deviation (from that revision's harness): the setup heap is frozen out of Python's cyclic collector (`gc.collect()`, `gc.freeze()`) before the warm-up. For the $2 \to 2$ to $2 \to 4$ rows a scratch launcher applied it to the unmodified harness of `1d27426b` (clean checkout); it is the protocol of `measure-g1-7leg`'s `de8d3eb8`, which the committed harness follows since that package's integration (on `loop` with this milestone), and the $2 \to 5$ rows ran it in the harness itself; `timing_protocol` in `data/g1.json` records it. The committed harness times with the whole generation heap in memory, which a runtime loaded from an artifact never holds, and full collections of that heap fell into the timed blocks: $u\bar{u} \to d\bar{d}g$ at $B = 128$ took 2.76 ms per point with the committed protocol (a collection of about 0.09 s in 2 of the 3 blocks), 1.95 ms with the frozen heap, 1.96 ms through the public `Runtime.load` executor in a fresh interpreter and 2.08 ms through `Runtime.evaluate` on decimal text; $gg \to ggg$ 22.9 against 20.6 ms. The evaluations' own collections stay in the timed blocks (0.2–2.8 % of the timed wall). Earlier campaigns (G-int and before) used the committed protocol.

Table 2: **Gate G1, full colour**, ms per point (gate statistic: median), finite part, one thread. Rows $B = 1$ and $B = 128$ (the Python-input $2 \to 4$ rows only at $B = 1$: their Python inputs took seconds per point). Multiplier and colour: time ÷ the fastest provider of the row (legend of Table 1). Grey rows: generation — pyAmpliCol generation plus plan export with its peak memory; MadLoop graph generation, export and compilation · the share of its loop-numerator coefficients in its time (Table 4); Recola process generation; OpenLoops: the prebuilt library and its load plus first call. MadLoop $u\bar{u} \to gggg$ was not exported: the $gg \to gggg$ export took 2.4 h here (HANDOFF §0.1), and OpenLoops binds the gate. Gate: upper line Python inputs (history), lower line native inputs (`loop`); the number is the time over the binding limit (MadLoop's time or 1.30× the faster of Recola and OpenLoops); **open**: the band contains the limit; **not gated**: the configuration runs below 5 ms per point at its largest batch (user rule of 2026-10-24, at every B; ratio as measured) (bands: Table 3).

Process	B	Python inputs[hist]	Native inputs, `loop`	MadLoop	Recola	OpenLoops	Gate
$gg \to gg$ $2 \to 2$, no gate	1	**140** ×69	**15.4** ×7.6	**2.02** fastest	**10.0** ×5.0	**3.20** ×1.6	no gate
	128	**132** ×66	**1.56** ×0.77	**2.02** fastest	**10.0** ×5.0	**3.20** ×1.6	no gate
	gen.	10.4 s · 0.3 GiB	9.04 s · 0.4 GiB	9.70 s · 64 %	0.142 s	ppjj · 6.80 ms	
$u\bar{u} \to d\bar{d}g$ $2 \to 3$	1	**40.6** ×27	**10.6** ×7.0	**3.17** ×2.1	**3.18** ×2.1	**1.52** fastest	**fail** ×21 **not gated** 5.39
	128	**32.4** ×21	**1.91** ×1.3	**3.17** ×2.1	**3.18** ×2.1	**1.52** fastest	**fail** ×16 **not gated** 0.97
	gen.	9.94 s · 0.3 GiB	8.96 s · 0.4 GiB	9.87 s · 73 %	0.406 s	ppjjj · 4.73 ms	
$u\bar{u} \to u\bar{u}g$ $2 \to 3$	1	**68.9** ×24	**18.4** ×6.3	**8.59** ×2.9	**5.55** ×1.9	**2.92** fastest	**fail** ×18 **not gated** 4.84
	128	**54.5** ×19	**2.65** ×0.91	**8.59** ×2.9	**5.55** ×1.9	**2.92** fastest	**fail** ×14 **not gated** 0.70
	gen.	10.4 s · 0.3 GiB	9.13 s · 0.4 GiB	10.9 s · 75 %	0.462 s	ppjjj · 8.43 ms	
$u\bar{u} \to e^-e^+g$ $2 \to 3$	1	*not generable*	**3.93** ×4.1	**0.948** fastest	**1.08** ×1.1	*no library*	— **not gated** 4.14
	128	*not generable*	**0.716** ×0.76	**0.948** fastest	**1.08** ×1.1	*no library*	— **not gated** 0.76
	gen.	CT coverage refused	10.7 s · 0.4 GiB	14.0 s · 79 %	0.701 s	—	
$u\bar{u} \to ggg$ $2 \to 3$	1	**215** ×17	**47.1** ×3.8	**28.2** ×2.3	**25.3** ×2.0	**12.4** fastest	**fail** ×13 **fail** ×2.9
	128	**179** ×14	**5.17** ×0.42	**28.2** ×2.3	**25.3** ×2.0	**12.4** fastest	**fail** ×11 **pass** 0.32
	gen.	12.3 s · 0.4 GiB	10.4 s · 0.4 GiB	17.6 s · 76 %	0.634 s	ppjjj · 23.7 ms	
$gg \to ggg$ $2 \to 3$	1	**2 760** ×28	**207** ×2.1	**247** ×2.5	**351** ×3.6	**97.0** fastest	**fail** ×22 **fail** ×1.6
	128	**2 560** ×26	**20.2** ×0.21	**247** ×2.5	**351** ×3.6	**97.0** fastest	**fail** ×20 **pass** 0.16
	gen.	27.9 s · 0.6 GiB	17.0 s · 0.5 GiB	47.0 s · 78 %	6.63 s	ppjjj · 159 ms	
$u\bar{u} \to e^-e^+gg$ $2 \to 4$	1	*not generable*	**43.5** ×4.3	**27.3** ×2.7	**10.2** fastest	*no library*	— **not gated** 3.27
	128	*not generable*	**4.98** ×0.49	**27.3** ×2.7	**10.2** fastest	*no library*	— **not gated** 0.37
	gen.	CT coverage refused	12.0 s · 0.4 GiB	24.8 s · 83 %	0.833 s	—	
$gg \to gggg$ $2 \to 4$	1	**101 000** ×36	**3 900** ×1.4	**38 700** ×14	**17 600** ×6.2	**2 840** fastest	**fail** ×27 **fail** ×1.1
	128	*not timed*	**397** ×0.14	**38 700** ×14	**17 600** ×6.2	**2 840** fastest	— **pass** 0.11
	gen.	335 s · 4.1 GiB	122 s · 1.8 GiB	8 680 s · 51 %	2 130 s	ppjjjj · 4 950 ms	
$gg \to t\bar{t}gg$ $2 \to 4$	1	**7 310** ×15	**825** ×1.7	**5 450** ×12	**1 970** ×4.2	**472** fastest	**fail** ×12 **fail** ×1.3
	128	*not timed*	**99.4** ×0.21	**5 450** ×12	**1 970** ×4.2	**472** fastest	— **pass** 0.16
	gen.	62.5 s · 0.9 GiB	31.5 s · 0.7 GiB	218 s · 64 %	61.4 s	ppttjj · 821 ms	
$u\bar{u} \to gggg$ $2 \to 4$	1	**5 760** ×15	**811** ×2.2	*not timed*	**908** ×2.4	**374** fastest	**fail** ×12 **fail** ×1.7
	128	*not timed*	**79.0** ×0.21	*not timed*	**908** ×2.4	**374** fastest	— **pass** 0.16
	gen.	56.1 s · 0.7 GiB	32.0 s · 0.7 GiB	—	32.8 s	ppjjjj · 606 ms	

Process	B	Python inputs[hist]	Native inputs, `loop`	MadLoop	Recola	OpenLoops	Gate
$u\bar{u} \to e^-e^+d\bar{d}g$ $2 \to 5$	1	*not timed*	*not timed*	**132** ×3.7	**35.3** fastest	**51.0** ×1.4	— —
	128	*not timed*	**16.0** ×0.45	**132** ×3.7	**35.3** fastest	**51.0** ×1.4	— **pass** 0.35
	gen.	—	17.1 s · 0.5 GiB	58.6 s	6.73 s	pplljjj · 108 ms	
$u\bar{u} \to e^-e^+ggg$ $2 \to 5$	1	*not timed*	*not timed*	**1 420** ×5.7	**251** fastest	**346** ×1.4	— —
	128	*not timed*	**55.6** ×0.22	**1 420** ×5.7	**251** fastest	**346** ×1.4	— **pass** 0.17
	gen.	—	16.8 s · 0.5 GiB	146 s	13.4 s	pplljjj · 727 ms	
$gg \to t\bar{t}e^-e^+g$ $2 \to 5$	1	*not timed*	*not timed*	**3 040** ×7.5	**620** ×1.5	**404** fastest	— —
	128	*not timed*	**60.6** ×0.15	**3 040** ×7.5	**620** ×1.5	**404** fastest	— **pass** 0.12
	gen.	—	22.7 s · 0.6 GiB	165 s	27.7 s	pplltt̄j · 1 060 ms	
$u\bar{u} \to ggggg$ $2 \to 5$	1	*not timed*	*not timed*	**569 000** ≥×3.1	**≤ 182 000** (outside protocol, poles on: upper bound)	*no library*	— —
	128	*not timed*	**1 660** ≥×0.0091	**569 000** ≥×3.1	**≤ 182 000** (outside protocol, poles on: upper bound)	*no library*	— **pass** 0.0029
	gen.	—	368 s · 4.8 GiB	44 000 s	—	—	
$gg \to ggggg$ $2 \to 5$, no gate (no matched provider)	1	*not timed*	*not timed*	*not exported*	*not generated*	*no library*	no gate
	128	*not timed*	**6 970**	*not exported*	*not generated*	*no library*	no gate
	gen.	—	2 620 s · 23.0 GiB	—	—	—	

The providers run in production mode on the same stream: OpenLoops with `truepoles 0`, Recola without IR poles (one amplitude call per point), MadLoop at its production stability setting on exports built with MG5′s production flags. OpenLoops has no installed library with QCD corrections to $\ell^+\ell^-$ + 1 or 2 jets (`ppllj_ew` carries electroweak corrections only; existing provider builds only), so the V+jets gates bind on MadLoop and Recola. Providers have no batch API: one time per point serves every B. **Gate statistic:** the median per point on both sides — over pyAmpliCol's timed blocks of one B, and over the provider's timed points (the stream's first 16; 8 for MadLoop $gg \to gggg$, Recola $gg \to gggg$, OpenLoops $gg \to gggg$, Recola $u\bar{u} \to gggg$, MadLoop $u\bar{u} \to ggggg$ and Recola $u\bar{u} \to ggggg$). The provider times are the records of the measure-g1 campaign (23–24 October, on cores 129–345), not re-run since, and for the $2 \to 5$ rows the production-mode runs of package `measure-g1-7leg` (25 October, each provider with its production Collier cache: on for Recola's COLLIER 1.2.8 and OpenLoops, off for MadLoop's COLLIER 1.2.9 with its seven-leg cache defect; $u\bar{u} \to ggggg$ imported from the reference campaign's three points); pyAmpliCol ran on cores 130, 138, 156, 174, 178 and 189, one job per eight-core group sharing an L3 cache, inside the cores 176–191 reserved for this campaign (no SMT siblings; DECISIONS 2026-10-25 ~13:45; the $2 \to 5$ rows inside the cores 128–175 of package `measure-g1-7leg`), while other users' jobs loaded those cores at 10–60 %. The three $B = 128$ blocks of a process spread by at most 4.5 % (half the range over the median; $u\bar{u} \to d\bar{d}g$ 1.890, 1.915, 1.921 ms). The gate's uncertainty is pyAmpliCol's block spread (half the range of three blocks, the median's standard error from five) plus the provider's half-range over its repeated runs (8 %, the campaign's largest, for the single runs of Recola $gg \to gggg$, Recola $u\bar{u} \to gggg$, OpenLoops $u\bar{u} \to e^-e^+d\bar{d}g$, MadLoop $u\bar{u} \to e^-e^+d\bar{d}g$, OpenLoops $u\bar{u} \to e^-e^+ggg$, Recola $u\bar{u} \to e^-e^+ggg$, MadLoop $u\bar{u} \to e^-e^+ggg$, OpenLoops $gg \to t\bar{t}e^-e^+g$, Recola $gg \to t\bar{t}e^-e^+g$, MadLoop $gg \to t\bar{t}e^-e^+g$ and MadLoop $u\bar{u} \to ggggg$). A gate passes or fails only if the whole band does; otherwise it is **open** and the configuration is repeated once (plan §5.8 item 10). Every timed call returned all its points (failed points: 0); rescue is off by protocol (no checks, no escalation), so the rescue rate is 0.

Table 3: **The gate at every batch size** (native inputs on `loop`, full colour): time over the binding limit and its uncertainty band (cell colour: green pass, red fail, amber open, grey not gated: below 5 ms per point at $B = 128$, user rule of 2026-10-24, at every B of the process). $B = 1$ is timed with the default 16 lanes (a one-point call pays a 16-lane chunk); the one-lane column is the development campaign's (`39824f9b`, history: the public path always runs 16 lanes in binary64). This campaign timed $B = 1$ and $B = 128$ (no verdict was open, so no repeat: `repeats` in `data/g1.json`); ° grey: the earlier run of `39824f9b`, `4e9b5a9e` at that batch size, its median over the same binding limit with its time (no band; an earlier engine under the committed protocol; not a verdict of this campaign).

Process	$B = 1$, **16 lanes**		$B = 1$, **1 lane** (39824f9b)		$B = 8$		$B = 32$		$B = 128$	
$u\bar{u} \to d\bar{d}g$	**5.39**	5.32–5.46	**2.13**	2.10–2.16	*1.46°*	2.87 ms	*1.15°*	2.26 ms	**0.97**	0.95–0.99
$u\bar{u} \to u\bar{u}g$	**4.84**	4.76–4.91	**1.63**	1.59–1.67	*1.16°*	4.40 ms	*0.87°*	3.31 ms	**0.70**	0.68–0.72
$u\bar{u} \to e^-e^+g$	**4.14**	4.12–4.16	**1.61**	1.60–1.62	*1.07°*	1.01 ms	*0.85°*	0.805 ms	**0.76**	0.75–0.76
$u\bar{u} \to ggg$	**2.93**	2.85–3.00	**0.84**	0.81–0.86	*0.68°*	11.0 ms	*0.74°*	11.9 ms	**0.32**	0.31–0.33
$gg \to ggg$	**1.65**	1.64–1.65	**0.45**	0.45–0.45	*0.42°*	52.7 ms	*0.34°*	42.9 ms	**0.16**	0.16–0.16
$u\bar{u} \to e^-e^+gg$	**3.27**	3.25–3.30	**0.78**	0.78–0.78	*0.62°*	8.20 ms	*0.42°*	5.54 ms	**0.37**	0.37–0.38
$gg \to gggg$	**1.06**	1.04–1.07	**0.31**	0.30–0.31	*0.30°*	1 120 ms	*0.26°*	977 ms	**0.11**	0.10–0.11

Process	$B = 1$, 16 lanes		$B = 1$, 1 lane (39824f9b)		$B = 8$		$B = 32$		$B = 128$	
$gg \to t\bar{t}gg$	**1.34**	1.33–1.36	**0.37**	0.36–0.37	*0.35°*	215 ms	*0.28°*	172 ms	**0.16**	0.16–0.17
$u\bar{u} \to gggg$	**1.67**	1.64–1.70	**0.43**	0.43–0.44	*0.47°*	230 ms	*0.37°*	178 ms	**0.16**	0.15–0.17
$u\bar{u} \to e^-e^+d\bar{d}g$	—		—		—		—		**0.35**	0.32–0.38
$u\bar{u} \to e^-e^+ggg$	—		—		—		—		**0.17**	0.16–0.18
$gg \to t\bar{t}e^-e^+g$	—		—		—		—		**0.12**	0.11–0.12
$u\bar{u} \to ggggg$	—		—		—		—		**0.0029**	0.0027–0.0032

2.1 Where the time goes

Table 4: **Stage split**, ms per point, full colour, at the largest batch timed (Python-input $2 \to 4$: $B = 1$). Trees/Born/CT: the per-point inputs, timed in Python before G-int (history: trees, counterterm program, Born and counterterm colour projections, scalars); on `loop` the executor forms them and, since lc-perf (`88657260`), times them itself (its `trees` stage: programs, closures, interferences). Numerator: seeds, fused factors and the dressing of the open-loop objects (the pure numerator evaluation). Scatter: closures into NinjaRS lanes. NinjaRS: the reduction without the masters; masters: the master integrals (canonical arguments, cache, oneloopmaster). Glue: point parameters, carrier conversion, assembly and, where the inputs are timed, the rest of the Python call (all rows of this table). MadLoop (instrumented copy; its points less one, 3 for $gg \to gggg$): Born/counterterm HELAS calls, loop-numerator coefficients, reduction plus masters, residual (stability test, colour sums). Bar: shares, colours as the column heads.

Process		Trees, Born, CT	Numerator	Scatter	NinjaRS	Masters	Glue	Total	Shares
$gg \to gg$	native, `loop`	0.201	0.866	0.0918	0.118	0.0378	0.236	**1.56**	
	Python^{hist}	128	0.851	0.0919	0.119	0.0381	2.96	**132**	
	MadLoop	0.0315	1.27	—	0.585		0.107	**1.99**	
$u\bar{u} \to d\bar{d}g$	native, `loop`	0.0925	0.754	0.465	0.140	0.114	0.343	**1.91**	
	Python^{hist}	28.1	0.749	0.462	0.137	0.111	2.66	**32.4**	
	MadLoop	0.0479	2.35	—	0.683		0.133	**3.22**	
$u\bar{u} \to u\bar{u}g$	native, `loop`	0.123	1.22	0.476	0.223	0.160	0.450	**2.65**	
	Python^{hist}	48.5	1.23	0.479	0.232	0.171	3.99	**54.5**	
	MadLoop	0.137	6.44	—	1.15		0.814	**8.54**	
$u\bar{u} \to e^-e^+g$	native, `loop`	0.0582	0.267	0.152	0.0269	0.0178	0.197	**0.716**	
	MadLoop	0.0184	0.754	—	0.133		0.0460	**0.951**	
$u\bar{u} \to ggg$	native, `loop`	0.461	2.93	0.477	0.459	0.216	0.600	**5.17**	
	Python^{hist}	168	2.92	0.476	0.459	0.225	5.75	**179**	
	MadLoop	0.346	18.5	—	2.54		2.87	**24.3**	
$gg \to ggg$	native, `loop`	4.18	12.0	0.542	2.05	0.493	1.02	**20.2**	
	Python^{hist}	2 530	11.7	0.519	2.03	0.483	10.5	**2 560**	
	MadLoop	2.42	184	—	11.3		39.5	**237**	
$u\bar{u} \to e^-e^+gg$	native, `loop`	0.176	2.85	0.655	0.443	0.250	0.586	**4.98**	
	MadLoop	0.289	22.7	—	2.38		2.11	**27.5**	
$gg \to gggg$	native, `loop`	105	225	2.50	48.8	5.99	12.3	**397**	
	Python^{hist}	97 300	3 750	4.08	38.6	5.81	137	**101 000**	
	MadLoop	88.9	18 600	—	209		17 700	**36 500**	
$gg \to t\bar{t}gg$	native, `loop`	11.7	47.1	1.02	22.6	6.55	9.19	**99.4**	
	Python^{hist}	6 490	751	3.55	20.7	6.58	29.3	**7 310**	
	MadLoop	24.2	3 290	—	105		1 740	**5 160**	
$u\bar{u} \to gggg$	native, `loop`	9.71	49.3	1.86	9.40	2.95	5.79	**79.0**	
	Python^{hist}	4 910	806	2.01	9.64	2.99	24.1	**5 760**	
$u\bar{u} \to e^-e^+d\bar{d}g$	native, `loop`	0.540	8.68	1.91	1.86	1.27	1.57	**16.0**	
$u\bar{u} \to e^-e^+ggg$	native, `loop`	1.49	38.5	2.11	7.80	2.64	3.10	**55.6**	
$gg \to t\bar{t}e^-e^+g$	native, `loop`	2.70	38.3	0.792	10.5	4.00	4.24	**60.6**	
$u\bar{u} \to ggggg$	native, `loop`	201	1 130	9.12	224	43.0	65.2	**1 660**	
$gg \to ggggg$	native, `loop`	2 890	3 570	10.9	434	40.4	32.6	**6 970**	

Verdict of gate G1 (processes from $2 \to 3$; $2 \to 2$ without a gate).

- **Native inputs on loop** (G1 at 1d27426b) at $B = 128$: **pass** for all 9 gated processes, $u\bar{u} \to ggg$, $gg \to ggg$, $gg \to gggg$, $gg \to t\bar{t}gg$, $u\bar{u} \to gggg$, $u\bar{u} \to e^-e^+d\bar{d}g$, $u\bar{u} \to e^-e^+ggg$, $gg \to t\bar{t}e^-e^+g$ and $u\bar{u} \to ggggg$. **Not gated** (below 5 ms per point, user decision of 2026-10-24; ratios as measured): $u\bar{u} \to d\bar{d}g$ 1.91 ms (0.97 of its limit), $u\bar{u} \to u\bar{u}g$ 2.65 ms (0.70 of its limit), $u\bar{u} \to e^-e^+g$ 0.716 ms (0.76 of its limit) and $u\bar{u} \to e^-e^+gg$ 4.98 ms (0.37 of its limit). At $B = 1$ with 16 lanes no gated process passes (a one-point call pays a 16-lane chunk, Table 3); with one lane per call, in the development runs of 39824f9b (history), $u\bar{u} \to ggg$, $gg \to ggg$, $gg \to gggg$, $gg \to t\bar{t}gg$ and $u\bar{u} \to gggg$ passed.
- **Python inputs** history (the engine before G-int, G1 at 1c971289): failed every process it generated, at $B = 128$ at ×11–20 of the binding limit, the per-point Python inputs taking 87 %–99 % of its time (Table 4); at $B = 1$ at ×12–27, where $gg \to gggg$, $gg \to t\bar{t}gg$ and $u\bar{u} \to gggg$ were timed only. It did not generate $u\bar{u} \to e^-e^+g$ and $u\bar{u} \to e^-e^+gg$ (Furry zeros, WP3.5; Z fermion signs, WP3.2).
- **Fastest of all tools** (aspiration), native inputs at $B = 128$, within the uncertainty band: faster than every provider for $u\bar{u} \to u\bar{u}g$, $u\bar{u} \to e^-e^+g$, $u\bar{u} \to ggg$, $gg \to ggg$, $u\bar{u} \to e^-e^+gg$, $gg \to gggg$, $gg \to t\bar{t}gg$, $u\bar{u} \to gggg$, $u\bar{u} \to e^-e^+d\bar{d}g$, $u\bar{u} \to e^-e^+ggg$ and $gg \to t\bar{t}e^-e^+g$; faster than every provider with a production-mode time for $u\bar{u} \to ggggg$ (MadLoop; Recola has only an upper bound of its production time, ≲ 182 s); at parity with the fastest for none; slower than the fastest for $u\bar{u} \to d\bar{d}g$ (OpenLoops). NJet, not a matched provider, takes 4.58 s for its two-evaluation full-colour $gg \to gggg$ test (Table 13).
- Seven-leg rows ($u\bar{u} \to e^-e^+d\bar{d}g$, $u\bar{u} \to e^-e^+ggg$, $gg \to t\bar{t}e^-e^+g$, $u\bar{u} \to ggggg$, $gg \to ggggg$) are generated by the WP3.1 generator, on loop since G-int, and timed by G1 (package measure-g1-7leg: full and leading colour at $B = 16$ and 128); $gg \to ggggg$ has no matched provider timing (addendum F, H): reported without a gate.

Table 5: **Levers**, measured on the rows of Table 4. Gains are measured where a column says so; otherwise they are the stage's share, an upper bound.

Lever	Evidence	Expected gain	Owner
Native per-point inputs (WP3.1 I3–I6: trees, Born, counterterm scalars in the executor)	Python inputs 87 %–99 % of the Python-input time at $B = 128$ (history)	measured, Python ÷ native inputs at equal B: ×3.7–26 at $B = 1$, ×17–126 at $B = 128$ ($2 \to 3$)	done: G-int, on loop since 4e9b5a9e
Binary64 point inputs (the providers' input; the protocol passes the stream's exact rationals)	paired run on the loaded artifact of G-int (4e9b5a9e, history), $B = 128$: $u\bar{u} \to d\bar{d}g$ 1.68 against 2.29 ms, $u\bar{u} \to u\bar{u}g$ 2.94 against 3.39 ms	measured: ×1.37 and ×1.15 (information; not a G1 configuration)	none: moot under the 5 ms gate rule (user, 2026-10-24); the protocol keeps exact inputs
Compiled native trees (kernels as the tree engine's SymJIT planes, lanes batched)	native trees (with the Born and counterterm inputs; the executor's timer) 3 %–26 % of the time at $B = 128$; $gg \to gggg$ 26 %	up to the share; ×1.4 for the largest	in part on loop: SymJIT plane applications (7db32901), lane-chunked tree closures (88657260); next: the native tree program (A5, native-dag)
Lanes by batch in binary64 (plan_export.lanes_for: 16 lanes also for one point)	$B = 1$: one 16-lane chunk per call	measured: ×2.5–4.2 at $B = 1$ (16 lanes on loop ÷ 1 lane of 39824f9b, different builds)	executor policy (WP3.1 or wave 4)
Finite-only result path (sum the virtual natively, no per-family lane export, Python assembly only for checks and observables)	glue 3 %–28 % of the native-input time	up to the share (largest at $2 \to 3$ with quarks)	plan_export, executor (wave 4)
Lane-major closures (no scatter, no second copy into the reducer's batch)	scatter 1 %–24 %	up to the share	executor, reducer adapter (wave 4)
AVX2 / AVX-512 kernels (runtime selection)	numerator 37 %–69 %	×1.0–1.4 on the dressing (gate G0)	wave 4 (WP4.6)
NinjaRS and masters: parallel reduction, transposed reduction, stage-2 sharing, a cheaper master cache key	NinjaRS + masters 6 %–29 %; masters 11 %–45 % of the reduction, oneloopmaster alone 1 %–12 %: the cache bookkeeping costs more than the integrals	one thread: up to the share; threads: DESIGN §5.6	NinjaRS requests (DESIGN §11.5)

Below 5 ms per point. $u\bar{u} \to d\bar{d}g$, $u\bar{u} \to u\bar{u}g$, $u\bar{u} \to e^-e^+g$ and $u\bar{u} \to e^-e^+gg$ take 1.91, 2.65, 0.716 and 4.98 ms per point at $B = 128$; by the user's decision of 2026-10-24 a process below 5 ms has no blocking gate and its slowdown against the providers need not be pursued: optimisation effort goes to higher multiplicities. Their ratios stay in the tables ($u\bar{u} \to d\bar{d}g$: 1.91 ms against a limit of 1.97 ms, OpenLoops 1.52 ms × 1.30, 0.97 of it; measured below 5 ms: not gated (user 2026-10-24), band 0.95–0.99). The question whether G1 should pass the points as exact rationals (the protocol) or as the binary64 text the providers read is moot under this rule, and the protocol stays. For information only (not G1 configurations, no

verdict formed from them): on 1d27426b the public Runtime.evaluate of the loaded artifact takes 2.08 ms per point on 60-digit decimal text for $u\bar{u} \to d\bar{d}g$ at $B = 128$, against 1.96 ms for its executor on the exact points (Section 2, timing protocol); at G-int (4e9b5a9e, committed protocol) a paired run (one core, ten alternating rounds) gave 1.68 ms on the binary64 text against 2.29 ms on the exact points (×0.73; $u\bar{u} \to u\bar{u}g$: 2.94 against 3.39 ms, ×0.87).

2.2 Leading colour

Table 6: **Leading colour** (LC), ms per point, same protocol as Table 2; in each cell the full-colour (FC) time of the same configuration and batch, and LC/FC. No provider ratio: the providers time full colour. LC uses the committed QCD colour continuation (examples/loop/source_colour); the V+jets rows are timed at LC with native inputs only. This campaign timed LC at $B = 128$; ° grey: the earlier run of 39824f9b at $B = 1$ with its FC time (an earlier engine under the committed protocol; history).

Process	Python inputs[hist]				Native inputs, loop			
	$B = 1$		largest batch		$B = 1$		largest batch	
$gg \to gg$	*not timed*		*not timed*		*not timed*		**0.716**	FC 1.56 · 0.46
$u\bar{u} \to d\bar{d}g$	**40.5**	FC 40.6 · 1.00	**37.4**	FC 32.4 · 1.15	*5.25°*	FC 10.9 · 0.48	**1.28**	FC 1.91 · 0.67
$u\bar{u} \to u\bar{u}g$	*not timed*		*not timed*		*9.00°*	FC 19.1 · 0.47	**1.64**	FC 2.65 · 0.62
$u\bar{u} \to e^-e^+g$	*not timed*		*not timed*		*not timed*		**0.521**	FC 0.716 · 0.73
$u\bar{u} \to ggg$	**184**	FC 215 · 0.86	**171**	FC 179 · 0.96	*22.8°*	FC 50.6 · 0.45	**2.70**	FC 5.17 · 0.52
$gg \to ggg$	**2 350**	FC 2 760 · 0.85	**2 300** ($B = 32$)	FC 2 550 · 0.90	*91.1°*	FC 211 · 0.43	**8.27**	FC 20.2 · 0.41
$u\bar{u} \to e^-e^+gg$	*not timed*		*not timed*		*not timed*		**1.71**	FC 4.98 · 0.34
$gg \to gggg$	*not timed*		*not timed*		*2 070°*	FC 4 100 · 0.50	**158**	FC 397 · 0.40
$gg \to t\bar{t}gg$	*not timed*		*not timed*		*418°*	FC 840 · 0.50	**33.3**	FC 99.4 · 0.33
$u\bar{u} \to gggg$	*not timed*		*not timed*		*380°*	FC 860 · 0.44	**29.8**	FC 79.0 · 0.38
$u\bar{u} \to e^-e^+d\bar{d}g$	*not timed*		*not timed*		*not timed*		**6.13**	FC 16.0 · 0.38
$u\bar{u} \to e^-e^+ggg$	*not timed*		*not timed*		*not timed*		**10.4**	FC 55.6 · 0.19
$gg \to t\bar{t}e^-e^+g$	*not timed*		*not timed*		*not timed*		**11.1**	FC 60.6 · 0.18
$u\bar{u} \to ggggg$	*not timed*		*not timed*		*not timed*		**522**	FC 1 660 · 0.31
$gg \to ggggg$	*not timed*		*not timed*		*not timed*		**3 470**	FC 6 970 · 0.50

LC is cheaper in the loop stage (fewer families and skeletons) and, since lc-perf (88657260: LC-aware pruning of the closure blocks, native tree closures in lanes of points), also end to end: with native inputs at $B = 128$ LC/FC is 0.18–0.73 ($gg \to t\bar{t}e^-e^+g$ gains most, $u\bar{u} \to e^-e^+g$ least), so LC is faster than full colour on every row; at G-int it was 0.55–1.28, slower than full colour for $u\bar{u} \to ggg$, $gg \to gggg$, $gg \to t\bar{t}gg$ and $u\bar{u} \to gggg$ (committed protocol; Section 2). The native trees, with the Born and counterterm inputs, take a larger share at LC ($u\bar{u} \to ggg$: 11 % at LC, 9 % at full colour; $gg \to gggg$: 31 % at LC, 26 % at full colour), the lever of compiled trees (Table 5). The $2 \to 5$ rows (on loop since this milestone, measure-g1-7leg) follow: LC/FC 0.18–0.50 at $B = 128$ ($gg \to t\bar{t}e^-e^+g$ 0.18, $u\bar{u} \to e^-e^+ggg$ 0.19, $u\bar{u} \to ggggg$ 0.31, $u\bar{u} \to e^-e^+d\bar{d}g$ 0.38, $gg \to ggggg$ 0.50). Generation is not cheaper at LC at seven gluons: the stored LC plan of $gg \to ggggg$ keeps all 4 707 full-colour loop families, where every lower gluon multiplicity halves them (154 ⟶ 77 and 1 100 ⟶ 550 at five and six gluons), and generates in 40.6 min against 37.9 min at full colour (LoopSlice.generate alone; Section 4). The cause, the LC degrees and pruning of the cells or the particle_veto path of this top-free generation, is under investigation (native-cells; generation-scale-2 if it is the particle_veto path; DECISIONS 2026-10-25 ~20:45).

2.3 Throughput, model recalibration and schedule M

Table 7: **Kernel throughput and the design recipe** (native inputs, full colour, largest batch). Work: the executor's modelled complex multiply-adds of the chosen kernel forms per point (one lane); throughput: work × lanes over the dressing time (fused-factor builds excluded). Native loop: seeds, factors, dressing, scatter and reduction; outside: everything else of the call (trees, glue). Recipe: DESIGN §1.3, finite only, where the design has a row; measured ÷ recipe for the total and for the NinjaRS stage (masters included) against its model R.

Process	Work, MCMAC/pt	Dressing, ms	G CMAC/s	Native loop, ms	Outside, ms	Total, ms	Total ÷ recipe	NinjaRS ÷ R
$gg \to gg$	0.932	0.718	1.30	1.11	0.451	**1.56**	1.0–3.1	0.99–1.13
$u\bar{u} \to d\bar{d}g$	0.556	0.447	1.24	1.47	0.442	**1.91**	no row	—
$u\bar{u} \to u\bar{u}g$	1.09	0.863	1.26	2.08	0.574	**2.65**	no row	—
$u\bar{u} \to e^-e^+g$	0.168	0.132	1.27	0.464	0.252	**0.716**	no row	—
$u\bar{u} \to ggg$	3.18	2.45	1.30	4.08	1.09	**5.17**	no row	—
$gg \to ggg$	14.3	11.1	1.29	15.0	5.19	**20.2**	no row	—
$u\bar{u} \to e^-e^+gg$	2.89	2.29	1.26	4.20	0.779	**4.98**	no row	—
$gg \to gggg$	268	218	1.23	282	115	**397**	1.0–2.9	1.07–1.26
$gg \to t\bar{t}gg$	56.0	44.3	1.26	77.2	22.2	**99.4**	0.6–1.8	1.08–1.52
$u\bar{u} \to gggg$	56.7	44.9	1.26	63.5	15.5	**79.0**	no row	—
$u\bar{u} \to e^-e^+d\bar{d}g$	8.80	7.37	1.19	13.7	2.29	**16.0**	1.0–3.0	0.96–1.72
$u\bar{u} \to e^-e^+ggg$	47.8	36.5	1.31	51.1	4.54	**55.6**	0.8–2.3	0.81–1.12
$gg \to t\bar{t}e^-e^+g$	49.3	37.2	1.32	53.6	7.03	**60.6**	0.8–2.2	1.03–1.42
$u\bar{u} \to ggggg$	1 170	902	1.29	1 410	255	**1 660**	0.6–2.0	0.91–1.24
$gg \to ggggg$	4 060	3 350	1.21	4 050	2 910	**6 970**	0.9–2.6	0.47–0.56

Recalibration. The kernels run 1.19–1.32 G complex MAC/s per core on the real plans, the rate gate G0 measured on its synthetic programs (1.29–1.37); the dressing term of the recipe holds. The NinjaRS stage, masters included, lands on the design's model R (measured ÷ R 0.47–1.72, Table 7). What the recipe misses is the **outer factor**: it assumed trees, seeds and counterterms add 5–10 % (DESIGN §1.3), but with the current native inputs the total is 1.14–2.3 times dressing plus reduction at full colour (up to 4 at LC). Table 8 re-estimates every design row with these measured terms.

Table 8: **Design estimates recalibrated by G1** (`tools/loop_v2_costmodel.py --g1`), ms per point, finite only, one thread. Recipe: DESIGN Table 1a's finite-only band (outer factor 1.05–1.10). G1: the recipe's dressing term, the NinjaRS model R × 0.473–1.718 (the band this campaign measures, Table 7) and the measured outer factor of the row's class representative at $B = 128$ with native inputs (same colour where measured). Measured: G1′s own time where the row was timed. Gate: G1′s binding limit at the row's largest batch where G1 timed the providers (1.3× the fastest production provider, or MadLoop's time where it binds), else 1.3× the best production provider of DESIGN Table 1b (protocol of 20 October), with the headroom gate ÷ upper estimate. The outer factor shrinks with compiled trees (Table 5); until then these replace the Table 1a targets.

Design row	Col.	Recipe	Outer	Class	G1 estimate	Measured	Gate	Headroom
$gg \to gg$	FC	0.502–1.52	1.54	$gg \to gg$	**0.624–2.30**	1.56	—	—
$gg \to gg$	LC	0.204–0.542	2.21	$gg \to gg$	**0.349–1.21**	0.716	—	—
$gg \to gggg$	FC	139–409	1.43	$gg \to gggg$	**157–585**	397	3 690	6.31
$gg \to gggg$	LC	55.7–146	1.53	$gg \to gggg$	**63.6–231**	158	—	—
$gg \to ggggg$	FC	2 710–7 920	1.78	$gg \to ggggg$	**3 790–14 100**	6 970	—	—
$gg \to ggggg$	LC	1 070–2 830	1.36	$gg \to ggggg$	**1 080–3 980**	3 470	—	—
$u\bar{u} \to ggggg$	FC	827–2 630	1.41	$u\bar{u} \to ggggg$	**950–3 660**	1 660	569 000	155.49
$u\bar{u} \to ggggg$	LC	206–526	1.39	$u\bar{u} \to ggggg$	**206–790**	522	—	—
$u\bar{u} \to e^-e^+ggg$	FC	24.5–68.1	1.14	$u\bar{u} \to e^-e^+ggg$	**21.0–81.1**	55.6	326	4.02
$u\bar{u} \to e^-e^+ggg$	LC	3.34–8.31	1.48	$u\bar{u} \to e^-e^+ggg$	**3.56–13.4**	10.4	—	—
$u\bar{u} \to e^-e^+d\bar{d}g$	FC	5.34–16.5	1.36	$u\bar{u} \to e^-e^+d\bar{d}g$	**5.61–23.7**	16.0	45.9	1.94
$u\bar{u} \to e^-e^+d\bar{d}g$	LC	1.19–3.14	1.81	$u\bar{u} \to e^-e^+d\bar{d}g$	**1.57–6.36**	6.13	—	—
$gg \to t\bar{t}gg$	FC	54.8–156	1.33	$gg \to t\bar{t}gg$	**56.0–215**	99.4	613	2.85
$gg \to t\bar{t}gg$	LC	14.6–40.1	1.58	$gg \to t\bar{t}gg$	**17.7–65.9**	33.3	—	—
$gg \to t\bar{t}e^-e^+g$	FC	27.4–76.1	1.15	$gg \to t\bar{t}e^-e^+g$	**23.9–91.3**	60.6	525	5.75
$gg \to t\bar{t}e^-e^+g$	LC	4.06–11.0	1.54	$gg \to t\bar{t}e^-e^+g$	**4.76–17.7**	11.1	—	—

Table 9: **Schedule M against B** (native inputs, one thread), pure-gluon classes, the only classes where M merges seed entries (DESIGN §5.6). The schedule-dependent stages are seeds, factors, dressing, scatter and the NinjaRS reduction ("the reduction included", §1.6 item 5); the trees, Born and counterterm inputs and the glue are the same work in both schedules. M: the development campaign's runs (`39824f9b`, not repeated); B: the configuration on `loop` (`1d27426b`), a later engine (native tree closures and plane applications, frozen setup heap) on the same processes, so the ratios pair runs of different builds and campaigns (the gate tool pairs them; history for M): end to end at $B = 128$ they set the M time with the older, slower trees against this engine's B time and do not measure M; LC was re-timed at $B = 128$ only. M is adopted where M/B ≤ 0.85 on the schedule-dependent stages at every common batch size; ± is the timing uncertainty of the ratio.

Process	Col.	Stages M/B, $B = 1$	Stages M/B, $B = 128$	End to end, $B = 1$	End to end, $B = 128$	Adopt M
$gg \to ggg$	FC	**0.96** ± 0.00	**0.98** ± 0.04	1.01	2.50	no
$gg \to ggg$	LC	—	**0.91** ± 0.05	—	5.64	no
$gg \to gggg$	FC	**0.85** ± 0.01	**0.83** ± 0.07	0.93	2.28	no
$gg \to gggg$	LC	—	**0.75** ± 0.03	—	6.44	**yes**

B against M (decision, provisional until the trees are compiled). On the schedule-dependent stages M passes the 15 % rule for $gg \to gggg$ at LC (0.75; DESIGN §5.6 measured 0.75–0.80 on the dressing side), and not for $gg \to ggg$ at FC (0.96–0.98), $gg \to ggg$ at LC (0.91) and $gg \to gggg$ at FC (0.83–0.85). The same rows qualify as in the development campaign's paired runs, on which the decision was made (`data/g1.json` of `c3a7be09`). End to end the gain is diluted by the tree inputs, which both schedules share and which compiled trees will shrink (Table 5). With a quark line M's entries do not merge at all (M/B work 1.00, DESIGN §5.6; equal by test). **Decision (G1):** schedule M for pure-gluon classes at leading colour from $n = 6$; schedule B for every other class at $n \leq 6$, including pure-gluon full colour; M stays the planned default from $n \geq 8$. The seven-leg comparison of M with B is open (the schedule-M run of `measure-g1-7leg` stopped; G1 and Table 1 time schedule B).

Table 10: **Values at the timed points**: pyAmpliCol's binary64 finite part (native inputs on `loop`) against each provider's production-mode value on the common stream points (the providers read the binary64 decimal text of the exact points), largest deviation, metric $|a - b| \,/\, \max(|a|, |b|, |B|\ \alpha_s/2\pi)$; Recola's finite part converted with the universal double pole (it returns no poles in production). Worst point: the point of the row's largest deviation, evaluated again with one rotation and the reduction-direction check in binary64 and in double-double (DD): E is the binary64 estimate, DD the DD value's deviation from the closest provider (named). Campaign: the first three points against the committed references (`tests/fixtures/loop/required`; $gg \to ggggg$: NJet's ten points), fixture metric and tolerance, native inputs (`loop`) and Python inputs (history); ‡ points above the tolerance that the user accepted (below the table).

Process	Pts	vs MadLoop	vs Recola	vs OpenLoops	Worst point: E	DD: closest provider	Campaign, native	Campaign, Python[hist]
$gg \to gg$	16	1.8×10^{-12}	7.9×10^{-13}	8.0×10^{-13}	pt 4: 3.0×10^{-12}	9.4×10^{-14} OpenLoops	—	—
$u\bar{u} \to d\bar{d}g$	16	6.7×10^{-13}	3.3×10^{-10}	6.7×10^{-13}	pt 1: 7.4×10^{-12}	3.2×10^{-13} MadLoop	3.6×10^{-10}	3.6×10^{-10}
$u\bar{u} \to u\bar{u}g$	16	8.4×10^{-12}	7.2×10^{-11}	6.1×10^{-12}	pt 1: 1.0×10^{-11}	2.4×10^{-13} MadLoop	8.6×10^{-11}	8.6×10^{-11}
$u\bar{u} \to e^-e^+g$	16	1.3×10^{-13}	8.6×10^{-14}	no library	pt 0: 1.2×10^{-12}	7.5×10^{-15} Recola	—	—
$u\bar{u} \to ggg$	16	1.4×10^{-12}	1.5×10^{-12}	8.4×10^{-11}	pt 2: 8.0×10^{-13}	1.3×10^{-14} MadLoop	—	—
$gg \to ggg$	16	7.1×10^{-7}	7.6×10^{-7}	7.6×10^{-7}	pt 10: 8.9×10^{-6}	6.0×10^{-12} Recola	—	—
$u\bar{u} \to e^-e^+gg$	16	1.2×10^{-10}	5.3×10^{-11}	no library	pt 8: 2.5×10^{-9}	4.3×10^{-13} Recola	—	—
$gg \to gggg$	8	3.2×10^{-8}	2.4×10^{-7}	2.4×10^{-7}	pt 1: 4.7×10^{-6}	4.4×10^{-12} Recola	2.4×10^{-7} **above 2.0×10^{-8}**	8.7×10^{-8} **above 2.0×10^{-8}**
$gg \to t\bar{t}gg$	16	1.3×10^{-8}	6.7×10^{-9}	6.7×10^{-9}	pt 13: 6.0×10^{-8}	5.5×10^{-15} OpenLoops	1.1×10^{-11}	1.1×10^{-11}
$u\bar{u} \to gggg$	16	—	9.7×10^{-12}	2.0×10^{-10}	pt 9: 2.6×10^{-9}	1.9×10^{-13} OpenLoops	—	—
$u\bar{u} \to e^-e^+d\bar{d}g$	16	4.3×10^{-11}	5.7×10^{-11}	5.7×10^{-11}	pt 15: 1.6×10^{-9}	5.5×10^{-14} OpenLoops	9.0×10^{-13}	—
$u\bar{u} \to e^-e^+ggg$	16	9.1×10^{-11}	1.1×10^{-10}	1.1×10^{-10}	pt 8: 1.1×10^{-9}	7.7×10^{-14} OpenLoops	6.0×10^{-13}	—
$gg \to t\bar{t}e^-e^+g$	16	8.7×10^{-9}	4.4×10^{-9}	5.6×10^{-9}	pt 2: 3.4×10^{-8}	5.8×10^{-13} OpenLoops	5.6×10^{-9}	—
$u\bar{u} \to ggggg$	3	2.1×10^{-12}	2.9×10^{-10}	no library	pt 0: 1.9×10^{-11}	1.4×10^{-12} MadLoop	2.9×10^{-10}	—
$gg \to ggggg$	0	not generated	not generated	no library	—	—	4.3×10^{-8} **‡ pts 4, 8 accepted**	—

Reading the values. The Born agrees with every provider to 1.7×10^{-13} or better. For the finite part, the double-double (DD) value at each worst point (Table 10) separates the sources: it agrees with the closest provider to 6.0×10^{-12} or better everywhere. Where pyAmpliCol's binary64 estimate E exceeds the deviation — $gg \to ggg$, $gg \to gggg$ and $gg \to t\bar{t}gg$ (above 10^{-9}) and $u\bar{u} \to gggg$ (1.9×10^{-13} in DD against 2.0×10^{-10} in binary64 at point 9) — the deviation is pyAmpliCol's own binary64 loss, which E flags. The other deviations above 10^{-11} stay in DD and are the providers' own accuracy: Recola's at $u\bar{u} \to d\bar{d}g$ (3.3×10^{-10}), $u\bar{u} \to u\bar{u}g$ and $u\bar{u} \to e^-e^+gg$ (COLLIER's default target 10^{-8}, HANDOFF §7), OpenLoops' at $u\bar{u} \to ggg$ (8.4×10^{-11}) and MadLoop's at $u\bar{u} \to e^-e^+gg$ (1.1×10^{-10}); MadLoop's production values also stay off the DD values at the ill-conditioned gluon and top points (up to 2.4×10^{-7}). At the campaign points $gg \to gggg$ point 1 misses the fixture tolerance (2×10^{-8}) in binary64 on both paths, within its estimate $E = 4.7\times10^{-6}$; its DD value agrees with Recola to 4.4×10^{-12}. ‡ $gg \to ggggg$ against NJet: points 4 and 8 exceed the tolerance 2×10^{-8} in binary64 (3.7×10^{-8} and 2.2×10^{-8} from NJet's double-double values, at most 4.3×10^{-8} over all its references), **accepted (user 2026-10-25): O(1e-8) binary64, within the estimate, DD = NJet**: estimates E 1.3×10^{-6} and 2.5×10^{-7}, and pyAmpliCol's double-double values equal NJet's; the check keeps its tolerance (`passed: false`).

3 Required processes: references and development validation

The plan requires the $2 \to 5$ processes of Table 1 (plan §5.7), compared with MadLoop and Recola, and with OpenLoops where a public library exists. A reference provider is run only if its generation completes in about 45 minutes, or in a time comparable to pyAmpliCol's own generation of the process (HANDOFF §0.1, addendum F). Otherwise pyAmpliCol is validated by Ward and gauge identities, Catani poles, LC/NLC/FC and DD/AP consistency, stability checks, lower multiplicities and existing references.

Table 11: **Provider generation on this host and the 45-minute rule** (HANDOFF §0.1): the rows of Table 1 where a provider generation takes more than 45 minutes or did not finish. MadLoop: graph generation + source export + compilation; Recola: process generation; OpenLoops: prebuilt public libraries (no generation time). Records: `data/g1.json` (G1 rows) and `data/measured.json` (`provider_generation`, the reference runs of the $2 \to 5$ rows), through `data/summary.json`. Every other row of Table 1 generates within 45 minutes with every provider (longest: MadLoop 10.9 min for $gg \to t\bar{t}e^-e^+g$ (EW)).

Process	MadLoop	Recola	OpenLoops	45-minute rule
$gg \to gggg$	2.4 h (285 + 8 354 + 37 s)	35.4 min	prebuilt ppjjjj	**MadLoop over 45 min; results exist and are used**
$u\bar{u} \to ggggg$	12.2 h (5 793 + 38 040 + 122 s)	2.8 h	no public library	**MadLoop, Recola over 45 min; results exist and are used**
$gg \to ggggg$	diagrams 17.4 h; unfinished (last state: exporting)	unfinished (last state: running, 2026-10-20)	no public library	**stopped 22 Oct; not to be restarted (HANDOFF section 0.1)**

Table 12: **Native validation of the required set** DEV (work/v2-generation-scale before its integration into loop at 4e9b5a9e, DESIGN WP3.1: ninth increment (6f5a64cc; threads not recorded) for $u\bar{u} \to e^-e^+d\bar{d}g$, $gg \to gggg$, $gg \to ggggg$; eighth increment (0ea83af2; threads not recorded) for $u\bar{u} \to e^-e^+ggg$, $gg \to t\bar{t}e^-e^+g$ (QCD), $gg \to t\bar{t}gg$; seventh increment (fef92189; threads not recorded) for $u\bar{u} \to ggggg$): generation (LoopSlice.generate, then the plan export; peaks of each stage; shared host under load), then binary64 virtuals with native per-point inputs and the production counterterm route at the fixture points, against the references committed on that branch (tests/fixtures/loop/required; fixture tolerance 2×10^{-8}). Largest deviation over the references and points, metric of Table 1; the per-reference values are in data/measured.json. For $gg \to ggggg$ the references are NJet (double-double and two binary64 runs) and the Catani operator for the poles, top-free with $N_f = 5$; its estimate column is the rotation plus reduction-direction check per point.

Process	References	Born	Finite	$1/\varepsilon$	$1/\varepsilon^2$	Estimate E	Generation + export
$u\bar{u} \to e^-e^+d\bar{d}g$	ML, Rc, OL · 3 pts	1.9×10^{-14}	8.7×10^{-13}	2.3×10^{-13}	1.0×10^{-13}	1.2×10^{-12}, 2.8×10^{-12}, 1.1×10^{-12}	37.7 s 0.4 GiB
$u\bar{u} \to e^-e^+ggg$	ML, Rc, OL · 3 pts	2.3×10^{-14}	8.5×10^{-13}	4.6×10^{-13}	2.0×10^{-13}	—	36.7 s 0.5 GiB
$gg \to t\bar{t}e^-e^+g$ (QCD)	ML, Rc, OL · 3 pts	1.4×10^{-14}	5.7×10^{-9}	3.7×10^{-12}	5.3×10^{-12}	—	43.7 s 0.6 GiB
$gg \to t\bar{t}gg$	ML, Rc, OL · 3 pts	2.1×10^{-14}	1.1×10^{-11}	1.0×10^{-10}	1.6×10^{-11}	—	38.5 s 0.7 GiB
$gg \to gggg$	Rc, OL · 2 pts	4.7×10^{-14}	4.5×10^{-11}	1.2×10^{-11}	5.2×10^{-12}	1.5×10^{-10}, 1.4×10^{-10}	154 s 2.2 GiB
$u\bar{u} \to ggggg$	ML, Rc · 3 pts	1.7×10^{-13}	2.9×10^{-10}	6.1×10^{-10}	4.4×10^{-11}	—	517 s 6.1 GiB
$gg \to ggggg$	NJet DD, NJet R1, NJet R2, Catani · 1 pt	8.1×10^{-13}	3.8×10^{-9}	1.5×10^{-9}	1.1×10^{-12}	2.0×10^{-8}	51.5 min 28.7 GiB

The $gg \to t\bar{t}e^-e^+g$ (QCD) finite part deviates by 5.7×10^{-9} at one of its three points and by at most 3.7×10^{-12} at the others: the point's known binary64 conditioning (earlier record, DESIGN WP3.1 fourth increment: ≤ 3.6×10^{-12} at two points; third point 5.7×10^{-9} in binary64 (estimate 1.7×10^{-9}), MadLoop to 1.9×10^{-14} in DoubleFloat). The electroweak target is in development (WP3.2: EW guards, the QED Catani operator, EW fermion lines, Feynman-gauge trees and the UFO's EW counterterms are on loop; the massive EW loop lines in Feynman gauge (EWI-4b, in progress), lepton-line signs (EWI-5) and EW native inputs are not). The $u\bar{u} \to ggggg$ row comes from DESIGN.md WP3.1 seventh increment (fef92189), the newest record of that process: the later re-runs on the merged heads (eighth and ninth increments) do not include it. It uses the same production counterterm route; its check is in Table 14.

4 Seven gluons: NJet and pyAmpliCol

4.1 NJet: Table 5 of arXiv:1209.0100, re-measured on this host

NJet is the independent reference for $gg \to ggggg$, where MadLoop and Recola cannot generate under the 45-minute rule and OpenLoops has no library. It is massless five-flavour QCD, and it is not one of the gate's matched providers.

Table 13: **NJet, s per point**, double precision, two evaluations each (NJet's scaling test). Paper: Xeon E3-1240 at 3.3 GHz, NJet 1.0. This host: one pinned core, idle at launch, of the shared host; 10 frozen points per channel, mean (spread ≤ 2 %); the multiplier is time ÷ the paper's. De-symmetrised sums equal the standard sums only after phase-space integration; they are timing figures, not point values.

Channel	Table 5	NJet 3.1.2		NJet 2.1.1		Table 5	NJet 3.1.2		NJet 2.1.1	
	standard full-colour sum					de-symmetrised sum				
$gg \to gg$	0.03	**0.00295**	×0.1	**0.00291**	×0.1	0.03	*no class*		*no class*	
$gg \to ggg$	0.22	*defective channel*		**0.141**	×0.6	0.07	*defective channel*		**0.0452**	×0.6
$gg \to gggg$	6.19	**4.59**	×0.7	**4.58**	×0.7	0.57	**0.39**	×0.7	**0.38**	×0.7
$gg \to ggggg$	171.31	**142**	×0.8	**246.7**	×1.4	3.07	**2.41**	×0.8	**4.19**	×1.4

At seven gluons one NJet 3.1.2 evaluation without the scaling test takes 71.2 s (standard) and 1.22 s (de-symmetrised; means over 10 points), at 69 MB; double-double costs 1 990 s per point. NJet 3.1.2's numerical five-gluon channel is 9/2 too large (NJet 2.1.1 is used there), and at the unrotated seven-gluon points NJet keeps only 4–6 digits in double precision (NaN at 3 of 10), so the reference values are taken at exactly rotated points.

4.2 pyAmpliCol at seven gluons DEV

On the WP3.1 branch before its integration (G-int; Table 12) the full $gg \to ggggg$ virtual, with the counterterms of the recursion and native per-point inputs, top-free with $N_f = 5$, agrees with NJet's double-double values to 3.8×10^{-9} in the finite part (per point: 3.8×10^{-9} over all references, point 0, ninth increment) and to 1.1×10^{-12} in the double pole; each point's rotation plus reduction-direction estimate bounds its difference. Generation (ninth increment) took 44.2 min at 14.7 GiB and the plan export 7.3 min at 28.7 GiB (peak RSS of each stage; 160 MB of plan JSON; the whole validation run, point evaluations included, peaked at 38.6 GiB), inside addendum H's budget (hours, below 150 GB). DESIGN

WP3.1 attributes the cost: 325 s before the counterterm recursion, 1,122 s with its counterterm program, 2,766 s once poles-uvir's always-generated companion merged (eighth increment), and 2,652 s after the ct-simplify merge (ninth increment, this record), 1,843 s of it the companion's seed table, which builds generic seeds (making it lean is open). A point with its rotation and reduction-direction checks took 82.3 s, a single-point development evaluation (threads not recorded): not a batched one-thread time and not comparable with NJet's 71.2 s per evaluation on one pinned core. The batched one-thread timing is G1′s (Table 1, Section 2), and so is the check of the timed configuration against NJet's ten points (Table 14).

4.3 Validation without a matched provider

Table 14: **Checks of the seven-leg QCD results.** First and third rows: G1′s checks of the timed configurations (engine 1d27426b on loop, native per-point inputs, full colour; binary64 finite part and Born, and the double pole of G1′s physical rows against the universal one); second row DEV on work/v2-generation-scale; Ward row WP2.5 on loop; deviation metric of Table 1. NJet values: fixture gg_ggggg_njet.json (NJet 3.1.2 double-double at exactly rotated points, scaling-test estimates ≤ 1.3×10^{-24}; points 0–2 are G1′s stream points 0–2 at NJet's scale).

Check	Result	Status
$gg \to ggggg$ full virtual against NJet (top-free, $N_f = 5$), 10 points (G1)	finite ≤ 3.7×10^{-8} against NJet's double-double values (median 2.0×10^{-9}; NJet's binary64 values ≤ 4.3×10^{-8}), Born ≤ 9.3×10^{-13}; every point within its estimate E; above the tolerance: point 4 3.7×10^{-8} ($E = 1.3\times10^{-6}$) and point 8 2.2×10^{-8} ($E = 2.5\times10^{-7}$), where the double-double values equal NJet's; double pole against the universal one ≤ 5.0×10^{-12} (points 0, 1, 2)	**accepted (user 2026-10-25): O(1e-8) binary64, within the estimate, DD = NJet** points 4 and 8; tolerance 2×10^{-8} unchanged, passed: false
Top-free full virtual against NJet at lower multiplicity	$gg \to gg$: ≤ 2.1×10^{-14} at 10 points, full virtual; $gg \to ggg$: ≤ 2.3×10^{-13} at the stable points; the flagged point 3×10^{-14} in DoubleFloat; $gg \to gggg$: 1.0×10^{-11} at point 0; point 1 1.6×10^{-7} in binary64 (estimate 2.7×10^{-6}), 4.7×10^{-15} in DoubleFloat (test_loop_v2_topfree.py)	passed
$u\bar{u} \to ggggg$ full virtual against MadLoop and Recola, 3 points (G1, the timed stream points)	finite: MadLoop ≤ 2.1×10^{-12}, Recola ≤ 2.9×10^{-10} (Recola's own: pyAmpliCol's double-double value lies 2.9×10^{-10} from it and 1.4×10^{-12} from MadLoop); Born ≤ 1.7×10^{-13}; double pole against the universal one ≤ 7.0×10^{-13}; the providers agree to 6.2×10^{-10} (reference merge)	passed (tolerance 2×10^{-8})
Ward identity, one gluon's $\varepsilon \to p$, at seven gluons	asymmetric Ward mode of WP2.5 (loop side only, bare + UV + R_2); passes on $gg \to gg$ and $u\bar{u} \to ggg$ (test_loop_v2_observables.py)	**pending — needs native Ward inputs at seven gluons (addendum H)**

5 Capabilities at this milestone

Table 15: **Capability matrix** of the v2 engine on loop at 4fc72e39, with branch work marked.

Feature	Status	Boundary
Recursive engine	supported	Trees, families and loop numerators from recursions; no loop diagram is enumerated (FeynKit graphs are test oracles only). Counterterms: see below.
Full, NLC and LC colour	supported	Exact seed tables in N_c; LC/NLC by generated classes and truncated cells. Since lc-perf LC is faster than full colour end to end on every G1 row at $B = 128$, the $2 \to 5$ rows included (LC/FC 0.18–0.73, Table 6); at seven gluons the LC plan keeps every full-colour family and generates slower than full colour (Section 2.2; investigation routed to native-cells).
Colour and spin correlations	supported	Directed colour histories at LC/NLC/FC, spin sources, resolved helicities, flavour aliases, Ward sources (WP2.5). Seven-leg directed LC/NLC tables need the Rust seed tables (WP3.1).
Per-order partials	supported	Every squared coupling order at tree level and at one loop, mixed Born orders, per-correction loop orders.
Poles at runtime	supported	poles = "all" | "none" | "uv" | "ir" | "split" (WP2.10, on loop since 24 October): finite-only mode with the finite part bitwise unchanged and the pole work skipped; "uv" and "ir" with the user's semantics of 23 October; the IR poles sum one Catani operator per correction, QED included (ew-poles, 8b52179b; a correction without a prediction gives none; the prediction for mixed QCD×EW sectors is still misleading, about 2 % in the single pole, open in WP3.2). The UV residue comes from an interim local module until the NinjaRS re-pin; a "uv" estimate needs a rotation check.
Stability estimates	supported	Rotation checks (any number, 0 = estimate absent), reduction direction, exact rescaling, in any combination; no automatic escalation (WP2.3). Generation's post-build validation re-checks a binary64 miss alone against a 32-digit reference: double-double must agree at the same tolerance, binary64 Borns at the tolerance and binary64 virtuals within the larger of the tolerance and their own estimate (one exact rotation and the reduction direction; an estimate above 10^{-9} or absent refuses), else generation refuses; an accepted point is logged (onshell-consistency, 4fc72e39).
Precision carriers	supported	Binary64, DoubleFloat, arbitrary precision and exact rationals in the native executor; since G-int the native per-point inputs run in binary64, DoubleFloat and arbitrary precision (DESIGN §5.5). Since 7db32901 binary64 runs SymJIT plane applications and needs no Symbolica licence; DoubleFloat and arbitrary precision run under pyAmpliCol's library key. Since precision-ap (a8409830) a master-integral invariant of one external leg is that leg's squared mass at the point's parameters: DoubleFloat and arbitrary-precision virtuals were O(1) wrong at binary64-rounded massive momenta and now agree with a MadLoop

Feature	Status	Boundary
		quadruple-precision run within 10^{-15} (and DoubleFloat massive spinors along $-z$ are fixed); a derived leg mass (e.g. M_W of loop_sm) is not bound yet (genericity M2, generation-scale-2).
Native executor, schedules	supported	Rust, 16 lanes (also for a one-point call, a G1 lever), threads; native tree closures in lanes of points with runtime AVX-512/AVX2 widths (lc-perf); G1 decision on the schedule-dependent stages (Table 9, provisional until compiled trees): M for pure-gluon LC from $n = 6$, B for every other class at $n \leq 6$; F as oracle. The NinjaRS reduction runs serially. Development report: stage times, modelled kernel work, master-integral time.
Plan v4 artifacts and API	partial	Published and loaded through Python, TOML and CLI; the artifact stores the native process (point_inputs = "native-v2"): a plain evaluation loads without any generation call. Since 7db32901 (addendum G) its SymJIT -O2 applications are stored as IR and recompiled at load for the loading CPU (AVX and FMA3 required), no host string is embedded, and binary64 values are bitwise equal after relocation to another machine, user and path; the parameter program's SymJIT bytes differ per compilation (a random external-call tag, values unaffected; upstream fix filed). Since model-determinism (4e9262b5) the compiled model, and with it the artifact's model member and digest, no longer depends on the process history. Since native-lowering (79f2a5bd, D phase 1 part 1) the plan JSON is written by the Rust plan model, byte-identical to the Python writer (kept as oracle); its bounded binary columnar encoding (23.0 MB against 193.6 MB of JSON at $gg \to ggggg$, full colour) becomes the artifact's plan member with generation-scale-2's writer. Correlated, resolved and flavour calls and the "uv"/"ir" selections still regenerate the slice on first use, and processes the native export refuses keep regenerated-at-load, silently (genericity M1: to become a typed, recorded refusal; generation-scale-2). The Rust/C/C++/Fortran SDK refuses v4 until then.
Counterterms (model UV, R_2)	supported	Plain tree vertices of the recursion inserted exactly once (WP2.2, a18b7599; addendum J, b4d76cc3), admitted by coupling orders and the public particle_veto; no witness table or loop matching in production. An unlowerable counterterm vertex that no tree of the request reaches is inert and recorded in the artifact; a reachable one is refused. Site closure is the test oracle. Per point they are formed natively (G-int), the Python stages kept as oracle; generation of the counterterm route has output-identical savings (ct-perf, 201bece5: compile memos, a content-addressed lowered-model cache), and since A4 (1b13005a) the native DAG builder is the production route for the loop blocks and the counterterm program (its counterterm emission byte-identical to the Python oracle; 11 listed features keep a recorded per-request Python fallback, any other native failure fails closed), at parity with the site oracle. The UFO's EW counterterms (ew-ct, 392d3f4a) are lowered through the model's own helper bodies with MadLoop's branch side for exactly real arguments: every Laurent piece of MadLoop's model library equals pyAmpliCol's to 3.7×10^{-15} in binary64 and 6.0×10^{-31} in double-double (against MadLoop's quadruple precision); the lowering takes 259 s cold (plan: 30 s, EWI-3R).
Direct R_2	deferred	A bonus by the user's rule; bounded controls only.
Electroweak, complex mass	in progress	WP3.2. On loop: typed refusals of EW requests with a non-zero width or an external photon (until the complex-mass scheme and $\alpha(0)$ per photon); the QED Catani operator (8b52179b); EW fermion lines (1d27426b): Dirac–Weyl EW vertices, W vertices along the fermion flow, conjugate readings, lepton- and colour-singlet-rooted processes, mixed-order Born with W exchange, the sites oracle equal to the recursion at W vertices; more than one colour-singlet fermion line refused until EWI-5; 't Hooft–Feynman-gauge trees with massive vector bosons and explicit Goldstones (EWI-4a, 6c87e276; Feynman = unitary and Slavnov–Taylor identities to 10^{-13}), which fixed two model-compiler defects that are also on main: a lone chiral projector had the opposite chirality (spinor-chain order in models/tensors.py, 3ae34543) and loop_qcd_qed_sm dropped the Goldstone flag of G^-; the UFO's EW counterterms (EWI-0, EWI-3, above). In development: Feynman-gauge loop lines (EWI-4b, in progress), lepton-line signs (EWI-5), EW native inputs (EWI-N), the EW ladder and the target; CMS later.
Required set	in progress	Seven of eight generate and agree with their references in all coefficients with native inputs (Table 12, validated on the WP3.1 branch before its integration), $gg \to ggggg$ against NJet's double-double values (its binary64 points 4 and 8 at $O(10^{-8})$ accepted by the user, within the estimate); the generator is on loop since G-int, and G1 times all seven in batches at full and leading colour (Section 2; the $2 \to 5$ rows since this milestone, measure-g1-7leg). The seven-gluon Ward check is pending — needs native Ward inputs. The EW target is in development (WP3.2). The provider reference trees no longer read the old working root (reference-relocate, 48aeb123; provider values bitwise, and the toolkit refuses provider files outside the configured reference root).
Performance gates	in progress	G1 on loop (Section 2, native per-point inputs, 1d27426b) at $B = 128$: 9 of 9 gated processes pass; 4 below 5 ms per point are not gated (user, 2026-10-24); $gg \to ggggg$ without a matched provider: no gate; $B = 1$ needs lane adaptation (Table 3). The Python per-point inputs before G-int failed every gate they generated (history). Missing: provider protocol-mode columns.
$gg \to 6g$	deferred	No $2 \to 6$ work until the $2 \to 5$ plan is complete (user, 23 October).

6 Architecture of the v2 engine

6.1 The computed quantity

For Born order a, loop order b and squared coupling order $s = a + b$,

$$V_{s,k} = \mathcal{N} \sum_{a+b=s} 2\,\mathrm{Re}\left[B_a^\dagger G_{BL} L_{b,k}\right], \quad k \in \{0, -1, -2\}, \quad V = C_\varepsilon \left(V_0 + \frac{V_{-1}}{\varepsilon} + \frac{V_{-2}}{\varepsilon^2}\right), \quad C_\varepsilon = \frac{(4\pi)^\varepsilon}{\Gamma(1-\varepsilon)},$$

where G_{BL} is the colour metric between Born and loop colour structures and $\mathcal{N}$ holds the initial-state averages and identical-particle factors (MadLoop/BLHA convention; couplings included, nothing divided by the Born). Directed colour and spin correlations replace $2\,\mathrm{Re}(\ldots)$ by the two cross terms $B_A^\dagger G L_B + L_A^\dagger G B_B$.

6.2 Pipeline

For every denominator family f, sector s and observable column o, v2 builds the four-dimensional numerator $N_{f,s,o}(q) = \sum_\alpha c_\alpha q^\alpha$ that NinjaRS reduces: Born-weighted and helicity-summed, produced by three recursions that share all work (Figure 2).

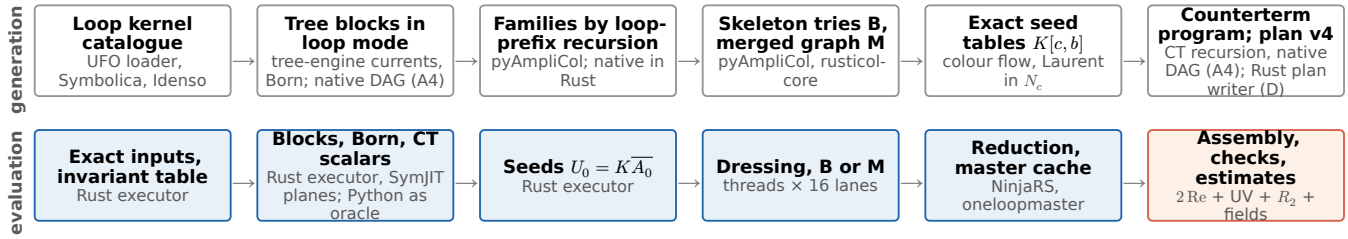


Figure 2: **v2 pipeline.** Upper lane: generation, once per process, into a plan v4 artifact. Lower lane: evaluation per batch of points. Blue: native batched numerics in the Rust executor (the per-point inputs since G-int); orange: Python per batch; white: generation. The owning code or dependency is named under each step.

Trees. The tree engine's colour-ordered Berends–Giele recursion computes, once per point, every off-shell current over every leg subset a loop vertex can absorb (the *blocks*), the Born colour components A_0 and, with WP2.2, every tree with one counterterm vertex. A loop-aware reachability test keeps exactly the currents some family position or root needs. Blocks with transverse sources use the exact transverse Yang–Mills kernels; loop segments, counterterm currents and Ward sources use the full ones.

Families and skeletons. A Recola-type recursion over loop prefixes (ordered leg sets, propagator species and masses) yields the denominator families, keyed by the canonical cyclic sequence of (leg mask, mass) and equal to Recola's tensor-integral identities. Per family it yields the colour-ordered *skeletons*: rings of blocks, each block summing all trees on its leg subset (Figure 3). Skeletons are never stored as a list; they are the leaves of the schedule tries. Only certified structural zeros are dropped (scaleless rings, external wave-function corrections, VEV tadpoles, colour zeros); nothing is restricted to 1PI.

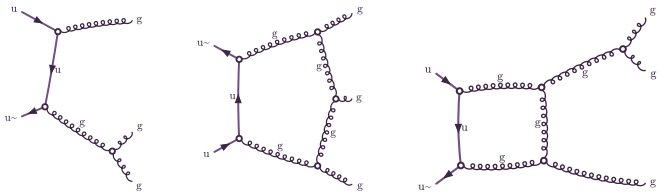


Figure 3: Three graphs of $u\bar{u} \to ggg$ drawn by FeynKit's renderer from its own generation (an oracle, not a production input). In v2 the Born (left) is a closure of blocks; the pentagon (middle) is a ring of five single-leg blocks; the box with a tree attachment (right) is a ring of four blocks, one of them the two-leg current, computed once per point and shared by every skeleton that contains that leg subset.

Loop numerators and schedules. An open-loop recursion dresses skeletons one segment at a time with polynomial-valued objects, $O' = O \cdot Y + \mathrm{raise}\big(O \cdot Z_\mu, q^\mu\big)$, in the NinjaRS monomial layout ($\binom{r+4}{4}$ coefficients at rank r, no rank cap). The seed is the colour-summed Born interference, and each block's helicities are summed as it is attached. One generator feeds three schedules (Table 16).

Table 16: **Three schedules on one generator** (DESIGN §2.2, §5.6).

	F: forward	B: suffix trie (default)	M: merged, primitive-seeded
Born weighting	at closure	at the leaves (one seed per skeleton class)	at the cut, one seed per colour ordering
Shared	prefixes across families	all prefixes with equal family and remainder	prefixes across families and compositions
Colour certificate	none needed	none needed	certified classes: adjoint rings, traces, quark rings
Role	oracle	default; generic fallback	pure-gluon LC from $n = 6$ (G1); planned default from $n \geq 8$

Colour. Colour lives only in exact seed tables $K[c,b]$, Laurent polynomials in N_c between seed classes c (the colour vector obtained by closing the cut of a skeleton's colour flow) and Born basis elements b; per point the seeds are $U_0 = \sum_b K[c,b]\overline{A_0[b]}$, with a symmetric-group FFT where certified. LC and NLC differ from full colour only in which classes

are generated and how cells are truncated: at LC each colour ordering is seeded by its own Born amplitude and skeletons below the leading N_c degree are never generated. Certificates (adjoint rings, fundamental traces, quark rings) are proved at loop level, never inherited from the tree level.

Reduction and masters. Families close directly into NinjaRS `TensorNumerator` lanes: one reduction per (family, sector, column), families with equal (propagators, rank) batched as lanes of one call, scalar masters from oneloopmaster cached per event. A second reduction in a relabelled frame reuses the numerators and the cached masters as the cheap stability check.

Counterterms. The model's UV and R_2 vertices, two-point ones included, are ordinary vertices of the tree recursion, and each contribution contains exactly one of them; external field factors are separate sources. Admission is the requested coupling orders, the exactly-once constraint and a public `particle_veto`: a coupling term whose loop particles include a vetoed particle is dropped, and a vetoed particle is removed from every tree vertex, loop line and propagator of the generation, so loops and counterterms stay consistent. Nothing is matched to the loop (user, 23 October; addendum J). On `loop` the counterterms come from the WP2.2 recursion (one counterterm order in the tree recursion, the default route, whose $gg \to ggggg$ program leaves no rule to site closure) with this admission (`ct-simplify`, b4d76cc3): no witness table and no loop matching. Since A4 (`1b13005a`) the native DAG builder emits this program (byte-identical to the Python builder, which stays as oracle) and builds the loop blocks. The UFO's electroweak counterterms (`392d3f4a`) enter the same way, lowered through the model's own helper bodies with MadLoop's branch side for exactly real arguments. A counterterm vertex that the model cannot lower and that no tree of the request reaches (the tree engine's own reachability, with the program's order limits and `particle_veto`) is inert and recorded in the artifact; a reachable one is refused. `particle_veto = [t]` replaces the internal top-free switch of Table 14; site closure and the witness machinery remain only as test oracles.

Precision, stability and assembly. Every kernel is generic over binary64, DoubleFloat, arbitrary-precision floats and exact rationals. A failed lane fails its point and never branches a kernel. The assembly takes $2\,\mathrm{Re}$ of $i/(16\pi^2)$ times the reduced family sum, plus local UV, R_2 and field terms, per sector and column, with optional rotation, reduction-direction and rescaling estimates and the finite-only mode. The public API selects `loop_engine = "v2"` from TOML, CLI or Python and stores plan v4 as a relocatable PACBIN member (zlib JSON of exact kernel term lists, written by the Rust plan model since `79f2a5bd`, no compiled code, digest checked at load) and the native process (`point_inputs = "native-v2"`: the tree program with the Born, counterterm and field components, their saved exact evaluators and the binary64 SymJIT applications of every tree kernel and of the parameter program, stored as IR and recompiled at load), so a plain evaluation loads without regenerating the process, on another machine, user and path too.

Table 17: **Asymptotic classes per event** for n gluons, full colour (DESIGN §1.4): $F_n \approx (n-1)!\left(\frac{1}{\ln 2}\right)^n/2$ families, 2^n helicities. The required set ($n \le 7$) is far below the crossover of schedule B with the unitarity class ($n \approx 30$–50); schedule M is planned as the default from $n \ge 8$.

Stage or code	Class
Tree blocks	$\Theta(n \cdot n!)$ currents; $\Theta(n! \cdot 2^n)$ current-helicity rows
Dressing, schedule B	$\Theta((n-1)! \cdot 4^n)$ seed injection; trie dressing dominates for $n \le 8$
Dressing, schedule M	$\Theta(n! \cdot 2^n \cdot n^4)$
NinjaRS	$\Theta(F_n)$ reductions, $\Theta(F_n \cdot n^8)$ work, $\Theta(4^n)$ distinct masters after caching
MadLoop (per diagram, per helicity)	$\Theta(D_n \cdot 2^n \cdot n^4)$, D_n super-exponential ($gg \to gggg$: 26 975 loop diagrams)
Unitarity with exact helicity sums (NJet)	$\Theta((n-1)! \cdot 2^n \cdot n^{6\ldots9})$

Table 18: **Ownership of the mathematics.** pyAmpliCol reuses its dependencies for algebra, tensors, graphs and integrals and owns the recursions and the assembly.

Component	Responsibility
ufo_model_loader	Lossless UFO loop-model import: Laurent-valued parameters and couplings, UV/R_2 categories, loop groups
Symbolica	Expressions, coefficient extraction in q, evaluators; exact rationals and arbitrary precision
Spenso, Idenso (Community)	Lorentz and colour tensors behind the kernel catalogue and the colour metrics
FeynKit (Community)	Test oracles: graph and family counts, overall factors, drawings (Figure 3)
NinjaRS, oneloopmaster	Integrand reduction with Laurent expansion; scalar masters in all carriers
pyAmpliCol tree engine	Blocks, Born, coupling orders and colour flow (Python lane; Rust recurrence)
pyAmpliCol loop v2	Families, skeletons, schedules, seed tables, counterterm program, plan v4, native executor, API

7 Sources and reproducibility

- **Build:** `typst compile benchmarks.typ benchmarks.pdf` in `docs/performance_reports/loop/` (Typst 0.15).
- **Status, in-flight work, decisions:** `data/milestone.json`, from the integration merges of 4fc72e39 since bae3b9ea (native-dag `1b13005a`, ew-gauge `6c87e276`, precision-ap `a8409830`, ew-ct `392d3f4a`, measure-g1-7leg `46f72f20`, reference-relocate `48aeb123`, model-determinism `4e9262b5`, native-lowering `79f2a5bd`, onshell-consistency `4fc72e39`; commit subjects "Integrate work/v2-…"); `workspace/reports/engine-v2/slice-status.md` (integration blocks, DECISIONS 2026-10-23/24/25/26, the milestone push m1025a of 2026-10-25 ~13:45 UTC, the m1025b attempt of 2026-10-26 ~00:50 UTC (pin `db13c260`, not pushed: its gate failed, DECISIONS ~01:35) and its replacement m1025c of ~06:35 UTC (pin `4fc72e39`, no G1 re-measure; the head check of the $2 \to 5$ rows on `db13c260`: slice-status UPDATE 2026-10-26 ~00:35, logs `workspace/tmp/integrate-measure-g1-7leg/hc/`); the user's gate decision of 2026-10-24, 16:05 UTC, is recorded in `data/milestone.json` `gate_decisions` and no longer applies after the re-measure; the user's rules of 2026-10-24, 22:35 UTC — uncoloured generation, green-to-red run times, summary pages with only Table 1, Figure 1 and legends, and the gate rule "below 5 ms: not gated (user 2026-10-24)", recorded as `gate_decisions.threshold_rule` and applied by `scripts/build_summary.py` and this report); `HANDOFF_24_10_2026.md` §6; MASTER_PLAN 2026-10-23/24 entries; REQUIREMENTS_ADDENDUM A, D, F, G, H, I, J; DESIGN §5.5, §5.7 (G1), §6.1, §6.2, §7.5, §12.1, §12.4, WP3.1, WP3.2; the 24 October pushes (59badc7a and b4d76cc3, gates in slice-status.md), the G-int push (79df367a, 25 October) and the m1025a push (bae3b9ea, 25 October). WP3.2 increments and dependencies: `workspace/reports/engine-v2/wp32/PLAN.md`; the EW counterterm comparison with MadLoop and measurement M1: `wp32/M1-ew-ct.md` and DESIGN §6.1; the genericity audit: `workspace/reports/engine-v2/genericity-audit/REPORT.md`; precision-ap's MadLoop comparison: `workspace/reports/engine-v2/precision-ap/`; reference-relocate's inventory and evidence: `workspace/reports/engine-v2/reference-relocate/`; model-determinism: DESIGN WP2.5; native-lowering: DESIGN §12.1 and WP3.1 (Rust migration D); onshell-consistency: DESIGN §11.1 and `docs/user/configuration.md`.
- **Table 1, Figure 1:** `data/summary.json`, written by `scripts/build_summary.py` from `data/g1.json`, `data/measured.json`, `data/milestone.json` and `tools/loop_benchmark/processes.json` by its main-configuration rule (the most recent measured configuration of each cell; a newer check that only repeats part of an older comparison does not replace it; revision ancestry from `data/milestone.json`); every entry names its configuration and source (README).
- **Table 11:** HANDOFF §0.1 (addendum F); the provider generation records of Table 1 (`data/summary.json`): `data/g1.json` for the G1 rows and the reference runs' records (`workspace/reference/runs/required/<process>/`, collected into `data/measured.json`) for the $2 \to 5$ rows; MadLoop's production-build times of the $2 \to 5$ rows: the same runs' `madloop-production-result.json`. **Table 12, Section 3, Section 4:** the native validations on `work/v2-generation-scale`, records `workspace/tmp/generation-scale/gint2/after/`, `gint/val1/` and `i7/val/` (the earlier seven-gluon points: `gint/val1/`), collected into `data/measured.json`; the earlier $gg \to t\bar{t}e^-e^+g$ record: DESIGN WP3.1 fourth increment. **Conventions:** MadLoop standalone, Recola, OpenLoops 2.
- **Section 2 (Table 2 to Table 10):** `data/g1.json` and the per-point values `data/g1-values.json`, written by `python -m tools.loop_benchmark gate` (README and docstrings of `tools/loop_benchmark`); streams `tools/loop_benchmark/streams/`; run records archived in `workspace/reports/engine-v2/measure-g1/records/` (first campaign `runs/`, fix round `runs2/`, the re-measure at G-int (`4e9b5a9e`) `gint/` with its paired binary64-input runs `confirm/<process>/ab.json`, and milestone m1025a's at `1d27426b` `m1025a/`: its runs, logs, the frozen-heap launcher `gc_launcher.py` and its protocol experiment `gc/`, and the public-path confirmation `confirm/uux_ddxg/` (write, load and evaluate records); provider records reused from the first campaign and the fix round, not re-run); the $2 \to 5$ rows' records (timings, stability, provider runs) in `workspace/reports/engine-v2/measure-g1-7leg/records/` (`runs-f2/`, `runs/m7-providers-*.json`), MadLoop exports of the G1-only processes in `workspace/reference/runs/g1/`; recalibration `tools/loop_v2_costmodel.py --g1 data/g1.json`. Builds: `loop-native-v2-slice-7db32901` (`loop`, `1d27426b`); history: `loop-native-measure-g1-genscale-39824f9b` (`work/v2-generation-scale` `889af02c` plus the timers of scratch commit `39824f9b`, tag `g1-native-inputs-39824f9b`: earlier runs, one lane, schedule M, stability) and `loop-native-measure-g1-033edf15` (Python inputs); G-int's earlier runs: `loop-native-v2-slice-4e9b5a9e`, release with one codegen unit and thin LTO; the harness is this branch's `tools/loop_benchmark` (the $2 \to 2$ to $2 \to 4$ rows: the harness of `1d27426b` with the frozen-heap launcher; the $2 \to 5$ rows: `measure-g1-7leg`'s harness, run worktree `2265a9f7` on that engine). Schedule M at seven legs: DESIGN §5.6.
- **Table 13** and NJet prose: `workspace/reports/engine-v2/njet-benchmark.md` §2, §5, §6 (raw `workspace/reference/runs/njet/timing/`; the seven-gluon single evaluations are read from those records into `data/measured.json`); NJet, arXiv:1209.0100 Table 5.
- **Table 14:** `data/measured.json` (`scripts/collect_milestone.py <workspace>`); top-free values at lower multiplicity: DESIGN WP3.1 and `test_loop_v2_topfree.py`; the seven-gluon generation history: DESIGN WP3.1 eighth and ninth increments. Memory in GiB = 2^{30} bytes.
- **Table 15, Section 6:** `docs/development/loop_v2/DESIGN.md` §1–§3, §5.5, §5.6, §6, §10, §12, REQUIREMENTS_ADDENDUM J (counterterm rule) and the integration records above.
- **History:** the legacy per-diagram engine's snapshot of 2026-10-18 (`data/measurements.json`, `provider-cold.json`, `legacy-labels.json`, `colour-measurements.json`) stays for the DESIGN citations and is no longer shown; its table is in the earlier report revisions (README).